

# AT命令操作

## AT 命令操作简介

本章节将通过 AT 命令控制KE1所使用的NB通信模组(BC28)进行联网和数据收发，进而简单的体验一下 AT 命令的使用方法和过程。

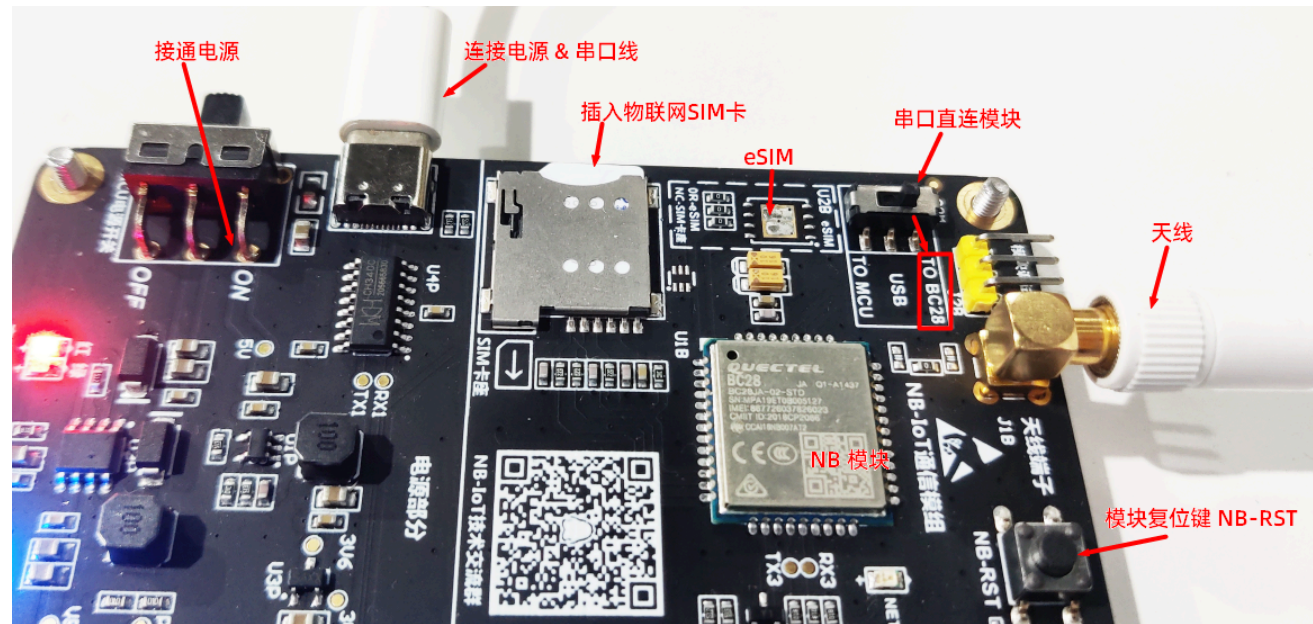
AT 命令的使用，请参考模组官方文档 "Quectel\_BC35-G&BC28&BC95 R2.0\_AT\_Commands\_Manual\_V1.3.pdf"，该文件在 [KE1综合实例源码](#) *KE1South* 包中

## 什么是 AT 命令

AT 命令（AT Commands）最早是由发明拨号调制解调器（MODEM）的贺氏公司（Hayes）为了控制 MODEM 而发明的控制协议。后来随着网络带宽的升级，速度很低的拨号 MODEM 基本退出一般使用市场，但是 AT 命令保留下来。查看 [百科](#)

在嵌入式开发中，经常是使用AT命令去控制各种通讯模组，比如ESP8266 WIFI模组、4G模组、NB模组等等。一般就是主芯片通过硬件接口（比如串口、SPI）发送AT命令给通讯模组，模组接收到数据之后回应响应的数据。

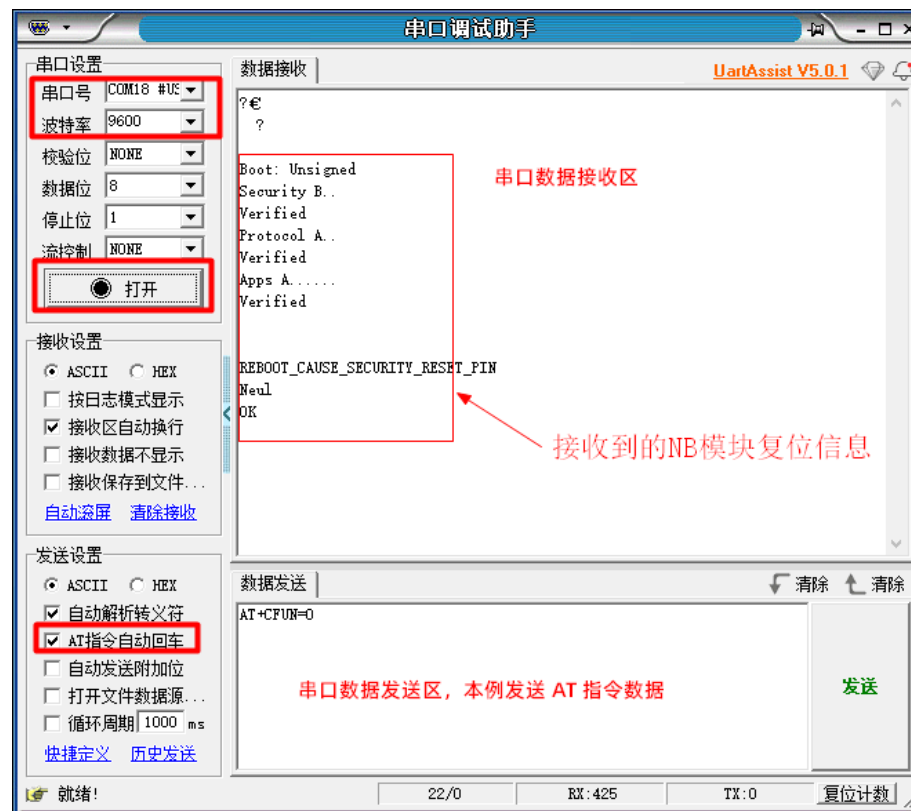
## 前期准备



1. 切换串口直连NB通信模组(TO BC28 )
2. 插入物联网 SIM 卡 (如果有焊接eSIM则不需要插卡)
3. 安装天线
4. 连接电源&串口线到您的电脑
5. 启动KE1电源
6. 启动串口工具

#### 串口工具使用帮助

1. 下载串口工具 [UartAssist串口调试助手](#)
2. 启动串口工具，设置串口号为KE1串口号（例如：COMx #USB-SERIAL CH340）
3. 设置串口工具波特率为 9600，其它参数默认即可
4. 勾选"AT 指令自动回车"
5. 点击"打开"按钮，连接串口。如果成功，即可发送 AT 命令控制模组
6. 按一下通信模组"手动复位(NB-RST)"键，查看是否能接受到模组的复位信息（可选）



## AT 命令连接云平台

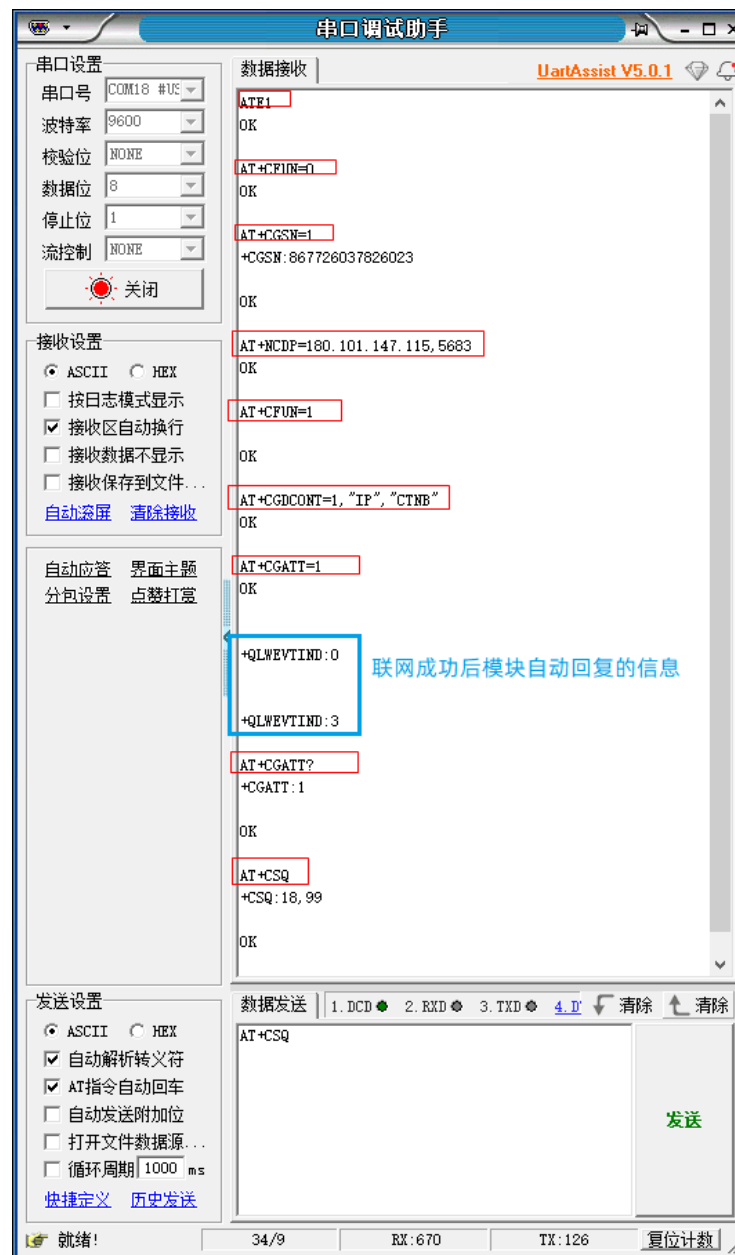
| AT 命令                        | 命令功能            | 命令响应                                | 说明                     | 错误响应原因                    |
|------------------------------|-----------------|-------------------------------------|------------------------|---------------------------|
| ATE1                         | 使能AT回显功能        | OK                                  | 发送的命令会返回               |                           |
| AT+CFUN=0                    | 模组关机            | OK                                  |                        |                           |
| AT+CGSN=1                    | 查询模组IMEI号       | +CGSN:867726037859099<br>(样例)<br>OK | 867726037859099即为IMEI号 |                           |
| AT+NCDP=180.101.147.115,5683 | 设置电信云平台CDP服务器地址 | OK                                  |                        |                           |
| AT+CFUN=1                    | 模组开机            | OK                                  | 需等几秒才有响应               | ERROR, 1.未安装SIM卡 2.未搜索到信号 |

| AT 命令                       | 命令功能     | 命令响应             | 说明                                      | 错误响应原因                                                     |
|-----------------------------|----------|------------------|-----------------------------------------|------------------------------------------------------------|
| AT+CGDCONT=1,"IP","","CTNB" | 设置PDP上下文 | OK               |                                         |                                                            |
| AT+CGATT=1                  | 开始联网     | OK               |                                         | ERROR, 1.未安装SIM卡 2.未搜索到信号 3.正在启动中                          |
| AT+CGATT?                   | 查询联网状态   | +CGATT:1<br>OK   | 0 表示未联网, 1 表示联网成功                       |                                                            |
| AT+CSQ                      | 查询网络信号强度 | +CSQ:18,99<br>OK | 18 为当前信号值, 该值范围0~31(含99表示无信号), 数值越大信号越好 | +CSQ:99,99 说明无信号。1. 未安装天线或SIM卡, 2.SIM卡余额不足, 3. 模组未使用AT命令启动 |

**注意：**模组不支持SIM卡热拔插，拔插需要重启设备。注意SIM插入方向！！切记

具体操作如下图所示，图中**红色**标识为我们发送的 AT 命令，**蓝色**数据为模组联网成功后自动回复的信息。





## AT 命令数据收发

使用 AT 命令发送数据时的注意事项:

1. 已通过本平台小程序或应用APP进行设备绑定注册，可参考【APP 应用开发入门】章节，完成模组云平台注册
2. 使用的NB模组 **已经联网，并已在云平台注册**，否则无法发送数据。
3. 发送的数据报文格式和内容格式需要按照云平台的定义进行收发。

- 云平台定义的数据报文格式如下：

|                   |                    |              |
|-------------------|--------------------|--------------|
| 数据类型1Byte (HEX格式) | 数据长度2Bytes (HEX格式) | 数据内容 (HEX格式) |
|-------------------|--------------------|--------------|

- 云平台定义的数据内容为JSON格式，[什么是JSON? | JSON在线编辑工具](#)

数据内容在发送时需要转换成HEX格式（16进制）。

例如把JSON数据内容 {"T":"10","H":"89"} 转换成HEX格式 7B2254223A223130222C2248223A223839227D 即 ASCII 转 HEX [在线转换](#)

| 数据类型 | 说明         |
|------|------------|
| 0x00 | 设备发送的数据    |
| 0x01 | 云平台下发的命令数据 |
| 0x02 | 设备响应命令     |

设备发送数据

AT 命令官方文档说明

### AT+NMGS Send a Message

|                                                                               |                                                                                                                       |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Write Command<br><b>AT+NMGS=&lt;length&gt;,&lt;data&gt;[,&lt;seq_num&gt;]</b> | Response<br><b>OK</b><br><br>If there is any error, response:<br><b>ERROR</b><br>Or<br><b>+CME ERROR: &lt;err&gt;</b> |
| Maximum Response Time                                                         | 300ms                                                                                                                 |

#### Parameter

|                        |                                                                                                                                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;length&gt;</b>  | Decimal length of message.                                                                                                                                                                                                                        |
| <b>&lt;data&gt;</b>    | Data to be transmitted in hex string format. The maximum length of data to be sent is 512 bytes.                                                                                                                                                  |
| <b>&lt;seq_num&gt;</b> | Sequence number. Range: 1-255. If non-zero <seq_num> is used to send CoAP data and there is CON or NON CoAP data with the same <seq_num> which has not been sent completely, the data to be sent will be discarded and an error will be returned. |

#### Example

```
AT+NMGS=3,AA11BB
OK
```

以下通过 AT 命令发送的数据仅仅是一个例子，可以参考例子自行修改发送不同的数据内容

数据发送 AT命令：

```
AT+NMGS=69,0000427B2254223A2232332E3132222C2248223A2236342E3733222C224C223A223231222C2253223A2232313037222C2256223A2234313430222C224E42223A2232
```

命令响应：+NNMI:4,AAAA0000

出错响应：

- ERROR 模组未联网
- +CME ERROR 模组未在云平台注册

发送AT命令参数说明：

- 69：发送的整个数据长度，本例发送 69 字节数据（1 Byte 数据类型 + 2 Bytes 数据长度 + 66 Bytes 数据内容）
- 00：数据类型（设备发送的数据）
- 0042：发送的数据长度(HEX 2Bytes)，0x0042 = 66
- 7B2254223A2232332E3132222C2248223A2236342E3733222C224C223A223231222C2253223A2232313037222C2256223A2234313430222C224E42223A223234227D  
：发送的数据是JSON：{"T":"23.12","H":"64.73","L":"21","S":"2107","V":"4140","NB":"24"}



## 接收云平台下发命令

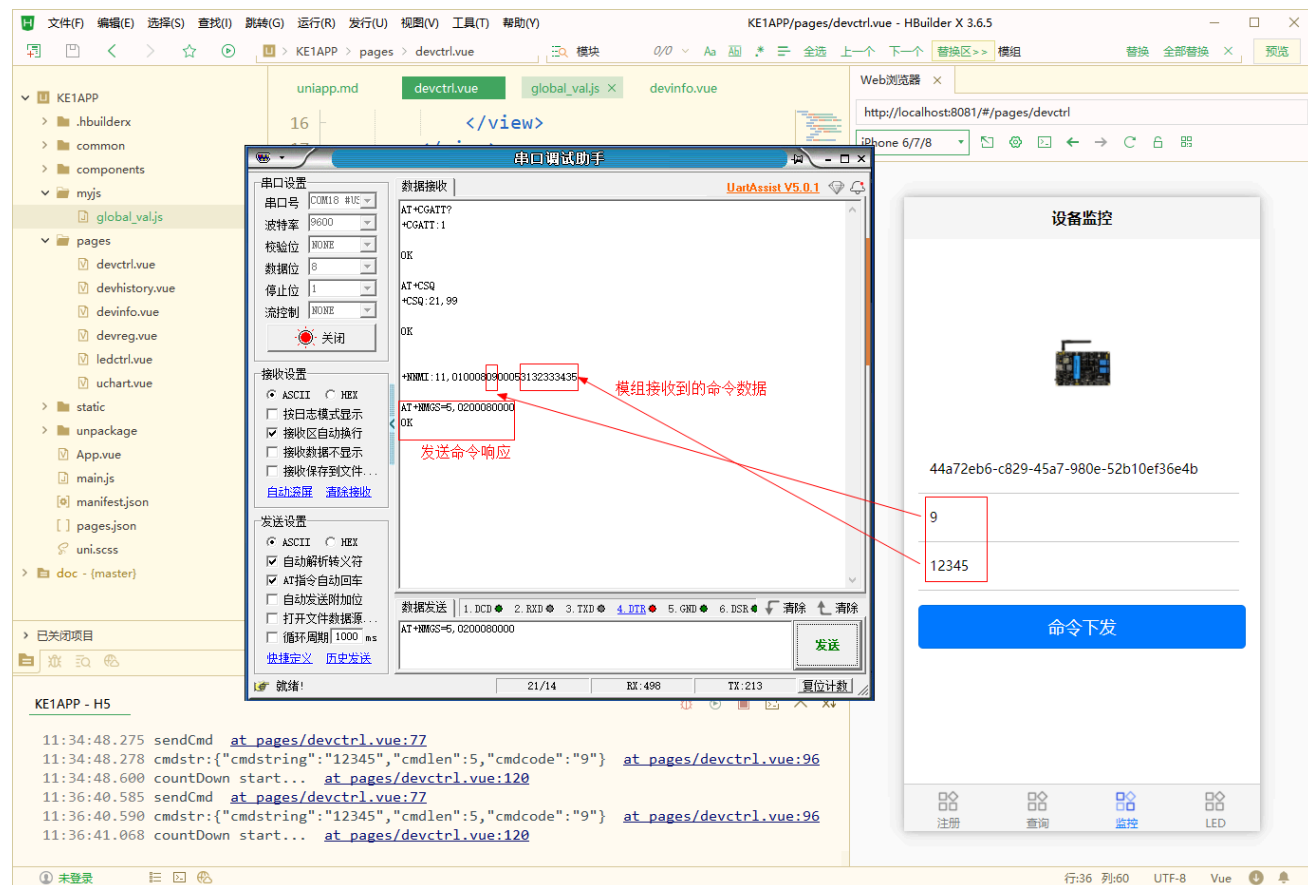
登陆本平台的微信小程序或者应用APP（可参考【APP 应用开发入门】章节），绑定设备后即可进行命令下发操作，操作成功后，模组将会自动收到如下信息

```
+NNMI:11,0100080900053132333435
```

响应参数说明：

- 11：收到的整个数据长度，本例收到 11 字节的数据
- 01：数据类型（云平台下发的命令数据）

- 0008：命令消息ID，该值会在云平台自动累加
- 09：自定义的命令码
- 0005：收到的命令数据长度(HEX 2Bytes)，0x0005 = 5
- 3132333435：自定义的命令数据HEX，3132333435(hex) = 12345 (ascii)



## 设备响应云平台命令

数据发送 AT命令：AT+NMGS=5,0200080000

发送AT命令参数说明：

- 5：发送的整个数据长度，本例发送 5 字节的数据
- 02：数据类型（设备响应命令）
- 0008：命令消息ID，该值需要跟下发的命令消息ID一致
- 0000：响应码(HEX 2Bytes)，默认为0x0000，即命令执行成功

本章节提供的AT命令可直接复制使用。请测试分析该样例：AT+NMGS=22,0000137B2254223A223132222C2248223A223936227D

## 物联网终端(STM32)开发

### STM32 物联网终端开发案例

本章节案例使用的是 ST 官方免费可视化开发工具 STM32CubeIDE (v1.11.0)，可先参阅章节【STM32CubeIDE 使用简介】熟悉开发工具的基本使用方法。

基于 STM32CubeIDE 可视化配置，可以快速学习使用 STM32 各类外设驱动，并结合NB物联网技术，模拟各类物联网典型应用的开发，适合物联网嵌入式终端零基础开发人员学习参考。

本章节所用电路原理图请参考【小熊座KE1开发板原理图.pdf】，该文件在[KE1综合实例源码](#) *KE1South* 源码包中。**所有例程仅着重说明外设驱动的配置参数和接口函数的使用，其它通用配置将不再赘述**

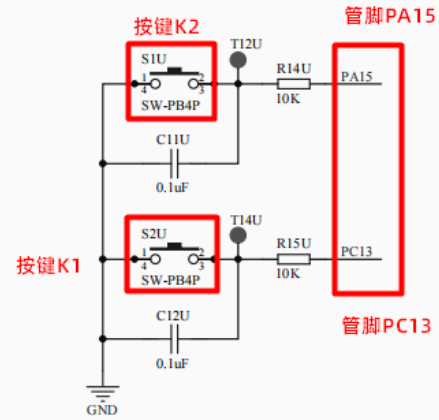
### GPIO 输入输出基础应用

物联网应用中，通常使用 GPIO 获取开关型传感器（人体热释红外传感器、倾斜开关、霍尔开关等）的状态事件，并通过 GPIO 控制继电器或LED状态灯，实现远程控制或告警等操作。

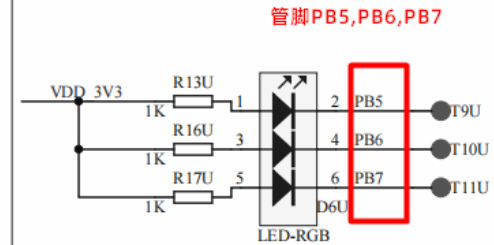
**本例目标：**通过读取按键 K1（管脚 PC13 中断模式），按键 K2（管脚 PA15 输入模式）状态，并控制三色LED灯（管脚 PB5，PB6，PB7 输出模式）的亮灭来学习 **GPIO 输入输出模式以及中断事件** 的基本用法（了解 **GPIO特性**、**中断的概念**）。

按键和LED灯的电路原理图如下：

## 扩展按键电路



## 三色LED灯电路



下载本例源码 [KE1\\_GPIO](#)

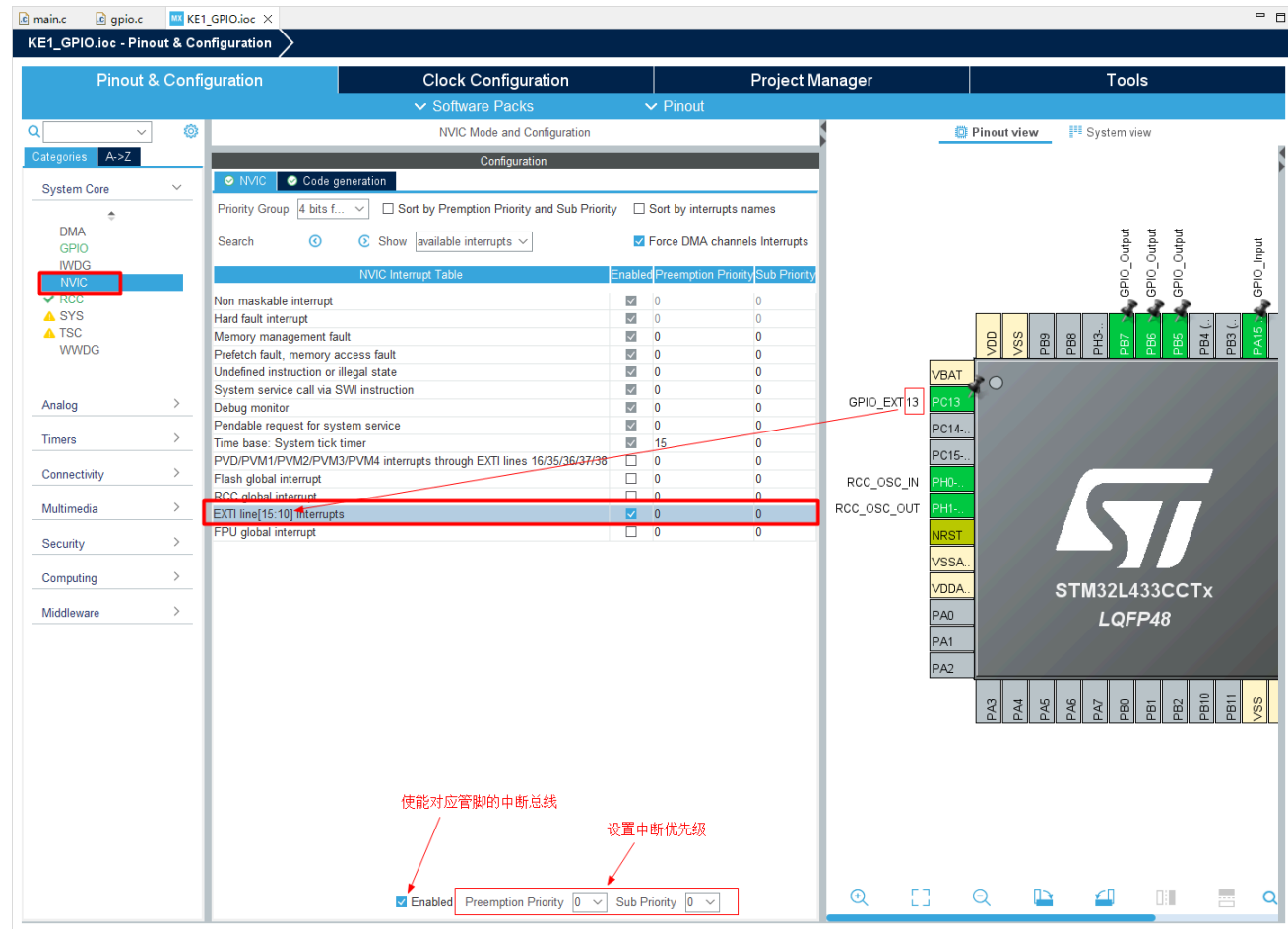
### 管脚功能配置

#### 1. 按键 K1 (GPIO-PC13) 设置中断模式

通过按键原理图可知，按键 K1 没有按下时，PC13管脚是悬空的，导致其输入状态未知。在 **System Core** (系统核心) **GPIO** (通用输入/输出接口) 中配置其电气属性 **Pull-up** (内部上拉)，使其在不按时有一个稳定的电平状态(默认为高电平)。按下时会被电阻下拉接地，从而输入低电平，这样在按下并松开按键时会产生一个上升沿波形，所以设置中断模式为 **External Interrupt Mode with Rising edge trigger detection** (上升沿中断模式)。当然您也可以尝试其它中断模式，对比一下不同点。







## 2. 按键 K2 (GPIO-PA15) 设置输入模式

按键 K2 的电路原理图与按键 K1 一样，本例需演示 GPIO 输入模式的用法，所以设置管脚 PA15 工作模式为 GPIO\_Input (输入模式)。按键按下时输入低电平，松开时为高电平。

main.c gpio.c KE1\_GPIO.ioc X

KE1\_GPIO.ioc - Pinout & Configuration

Pinout & Configuration Clock Configuration Project Manager Tools

Software Packs Pinout

GPIO Mode and Configuration

Configuration

Group By Peripherals

GPIO RCC NVIC

Search Signals Search (Ctrl+F) Show only Modified Pins

| Pin No.     | Signal on | GPIO output | GPIO mode     | GPIO Pull     | Maximum | Fast Mode | User Label | Modified |
|-------------|-----------|-------------|---------------|---------------|---------|-----------|------------|----------|
| PA15 (JTDI) | n/a       | n/a         | Input mode    | Pull-up       | n/a     | n/a       |            | ✓        |
| PB5         | n/a       | High        | Output Pu...  | No pull-up... | Low     | n/a       |            | ✓        |
| PB6         | n/a       | High        | Output Pu...  | No pull-up... | Low     | Disable   |            | ✓        |
| PB7         | n/a       | High        | Output Pu...  | No pull-up... | Low     | Disable   |            | ✓        |
| PC13        | n/a       | n/a         | External I... | Pull-up       | n/a     | n/a       |            | ✓        |

PA15 (JTDI) Configuration :

GPIO mode Input mode

GPIO Pull-up/Pull-down Pull-up

User Label 自定义管脚名称(将自动在代码中声明)

管脚属性配置

管脚工作模式：输入模式

管脚电气属性：内部上拉

设置管脚PA15工作模式

GPIO 输入模式

STM32L0

Reset\_State  
ADC1\_EXTI15  
LOD\_SEG17  
SPI1\_NSS  
SPI3\_NSS  
SWPMI1\_SUSPEND  
SYS\_JTDI  
TIM2\_CH1  
TIM2\_ETR  
USART2\_RX  
USART3\_DE  
USART3\_RTS  
GPIO\_Input  
GPIO\_Output  
GPIO\_Analog  
EVENTOUT  
GPIO\_EXTI15

### 3. 三色LED灯 (GPIO-PB5, PB6, PB7) 设置输出模式

三色LED灯控制管脚 PB5, PB6, PB7, 按照电路原理图设置工作模式为 GPIO\_Output (输出模式), 默认输出高电平 High。管脚输出高电平灯灭, 低电平灯亮。详细配置如下图

### 三色LED灯电路

管脚PB5,PB6,PB7

main.cKE1\_GPIO.loc ×

KE1\_GPIO.loc - Pinout & Configuration

Pinout & ConfigurationClock ConfigurationProject ManagerTools

Software PacksPinout

Pinout viewSystem view

Configuration

Group By Peripherals

GPIORCCNVIC

Search Signals

Search (Ctrl+F)

Show only Modified Pins

| Pin  | Signal | GPIO | GPIO   | GPIO    | Maxi    | Fast    | User | Modif |
|------|--------|------|--------|---------|---------|---------|------|-------|
| PA15 | n/a    | n/a  | n/a    | Input   | Pull-up | n/a     | n/a  |       |
| PB5  | n/a    | High | Output | No p... | Low     | n/a     |      |       |
| PB6  | n/a    | High | Output | No p... | Low     | Disable |      |       |
| PB7  | n/a    | High | Output | No p... | Low     | Disable |      |       |
| PC13 | n/a    | n/a  | Ext... | Pull-up | n/a     | n/a     |      |       |

PB5 Configuration :

GPIO output levelHigh

GPIO modeOutput Push Pull

GPIO Pull-up/Pull-downNo pull-up and no pull-down

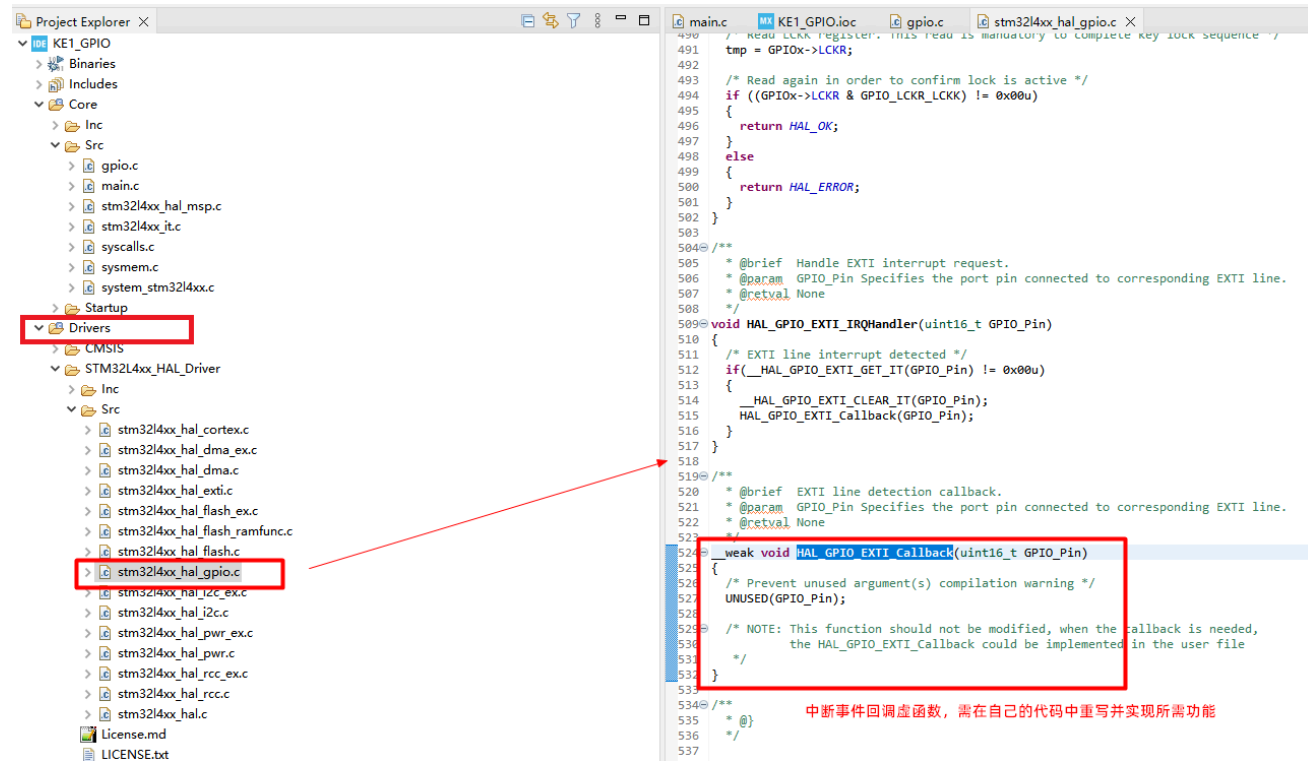
Maximum output speedLow

User Label

功能代码说明

1. 中断事件回调处理函数

MCU 在检测到中断事件时，会自动调用对应的中断函数，并运行用户代码。GPIO 中断回调函数的定义在驱动源码 "stm32l4xx\_hal\_gpio.c" 中，定义为一个 **弱函数** "\_\_weak"。如下图

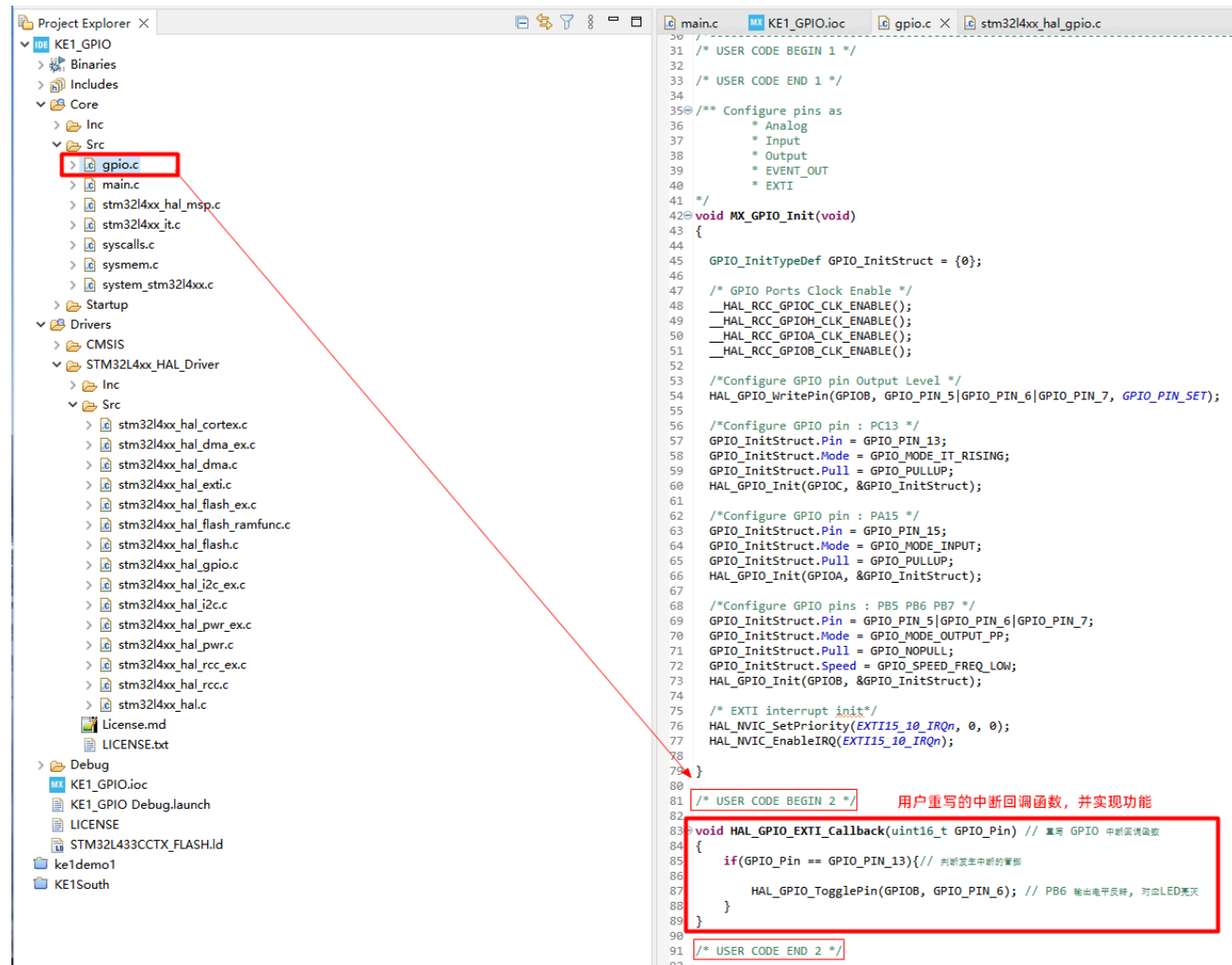


为实现自己的中断功能，我们需要在自动生成的用户代码文件 gpio.c 中重写该中断函数，并实现所需功能。本例功能是：检测到对应管脚有中断事件则改变 LED 灯(PB6)的状态(亮或灭)。

click

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) // 重写 GPIO 中断回调函数
{
    if(GPIO_Pin == GPIO_PIN_13){// 判断发生中断的管脚

        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_6); // PB6 输出电平反转，对应LED亮灭
    }
}
```



## 2. 读取管脚输入状态函数

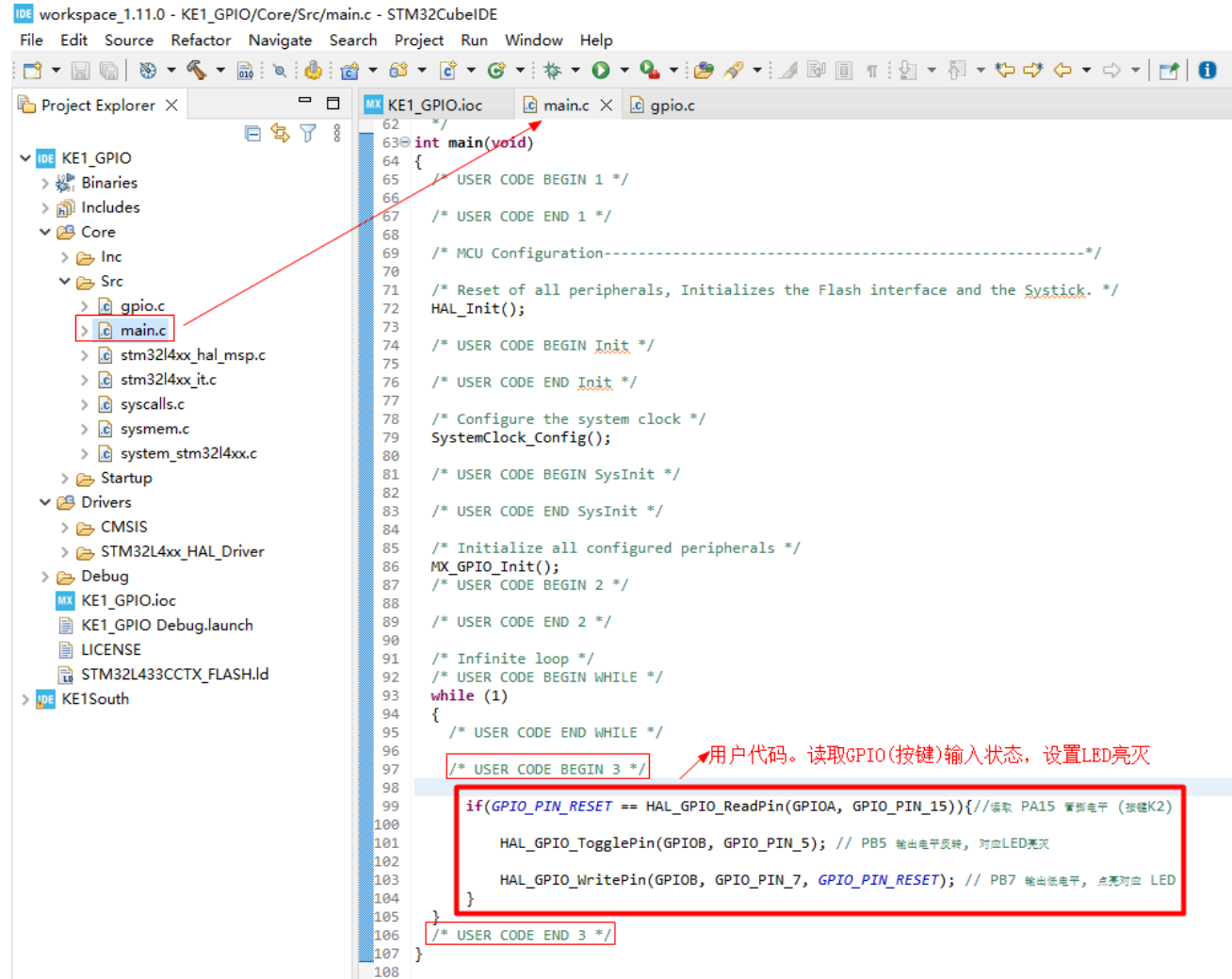
在主函数 main.c 文件的入口函数中，代码循环读取并判断按键 K2 是否按下（即管脚 PA15 输入电平是否为低），如果按下则改变LED灯状态，代码如下

```
if(GPIO_PIN_RESET == HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_15)){//读取 PA15 管脚电平（按键K2）
```

```
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_5); // PB5 输出电平反转，对应LED亮灭
```

click

```
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET); // PB7 输出低电平，点亮对应 LED
}
```



**注意：**一般情况下，按键被按下时会有抖动，所以判断按键是否有效按下时，用户代码需要有按键消抖逻辑，可自行 [百度搜索 STM32按键消抖](#) 并实现。

### 调试运行代码

编译并使用 ST-Link 调试运行代码，请参考【KE1 快速体验】章节

## UART 串口联网模组通信应用

**UART 串口** 是嵌入式应用开发中使用非常频繁的外设之一，主要用来跟其它串口设备通信，例如通过串口发送AT命令控制联网模块上网，或通过串口工具收发配置参数等。也经常用来输出代码运行信息进行 bug 调试和监控。

**本例目标：**使用串口1（UART1）输出代码调试信息，使用串口3（UART3）发送 AT 命令控制 KE1 联网模组 BC28 连接NB网，与云平台进行数据交互（电路原理图中 UART3 已连接通信模组 BC28 的串口）。

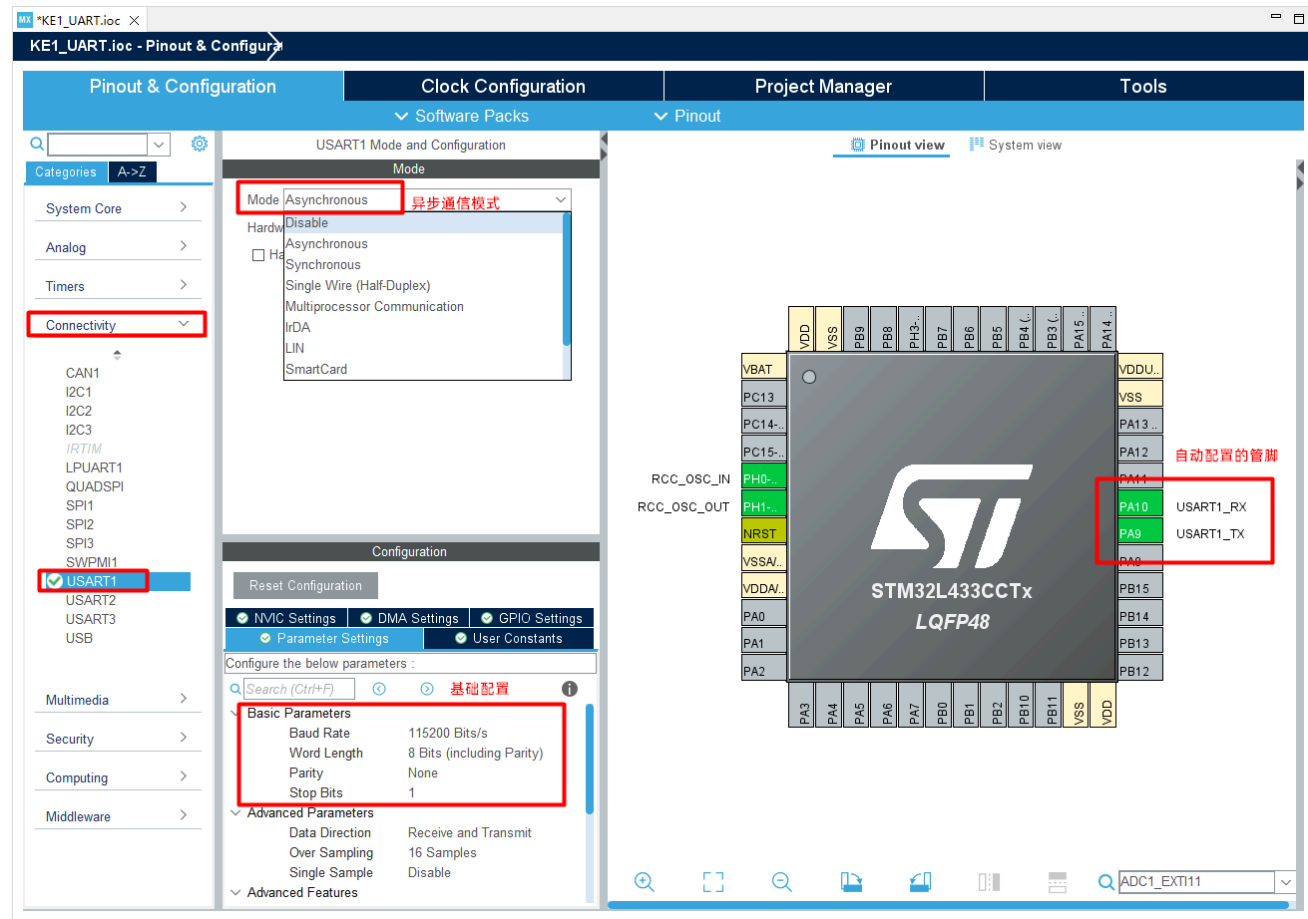
**注意：**在使用 KE1 开发串口应用时，请按下图初始化 KE1



### 管脚功能配置

#### 1. 配置串口 1（UART1）

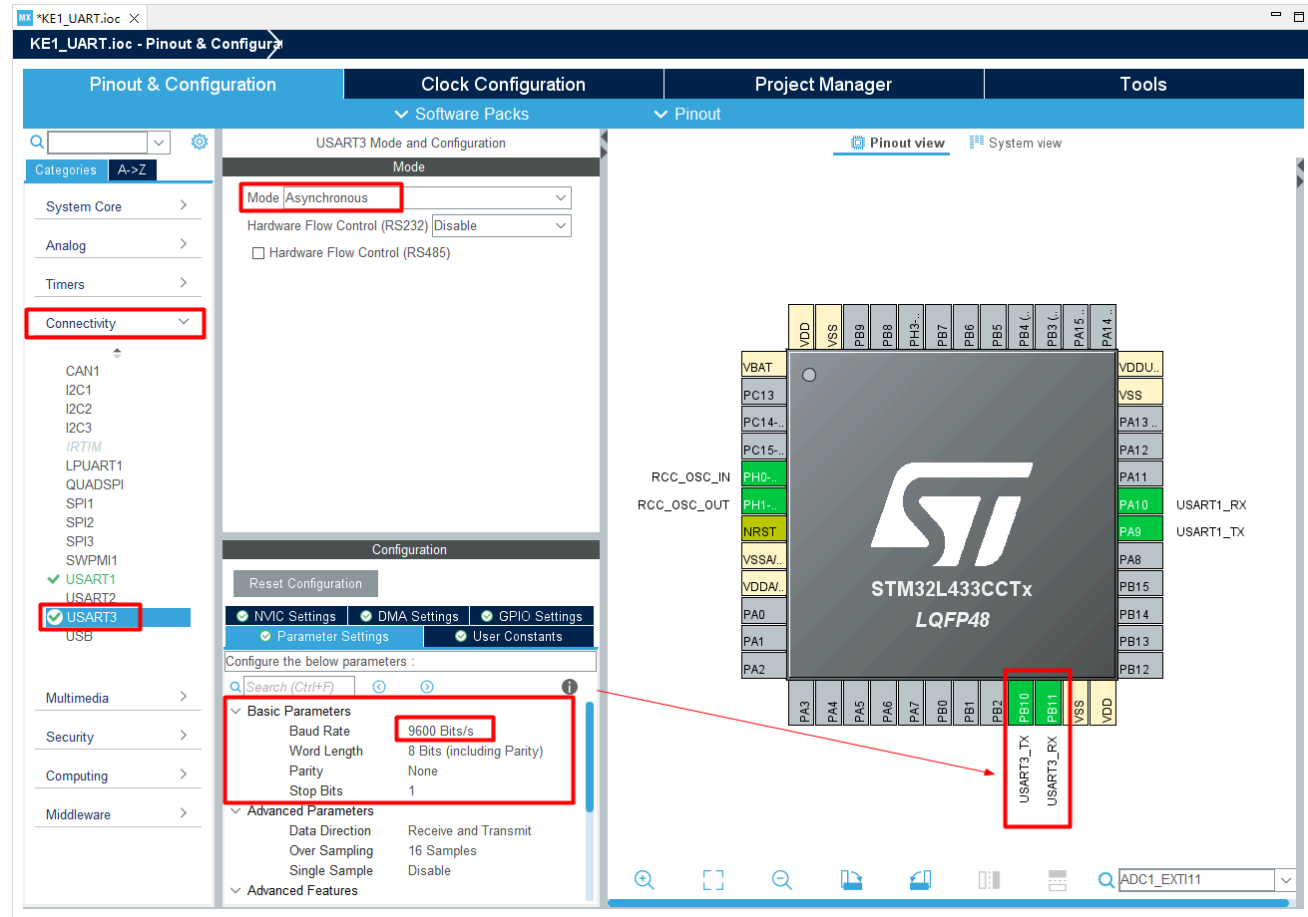
在 **Connectivity** (连接) **USART1** (通用同步/异步串行接收/发送器1) 中，配置工作模式为 **Asynchronous** (异步)，所占用的管脚为 PA9 (USART1\_TX 数据发送) PA10 (USART1\_RX 数据接收) 自动配置。工作波特率为 **115200**。UART1 用来输出代码运行调试信息，即作为debug调试口。



## 2. 配置串口 3 (UART3)

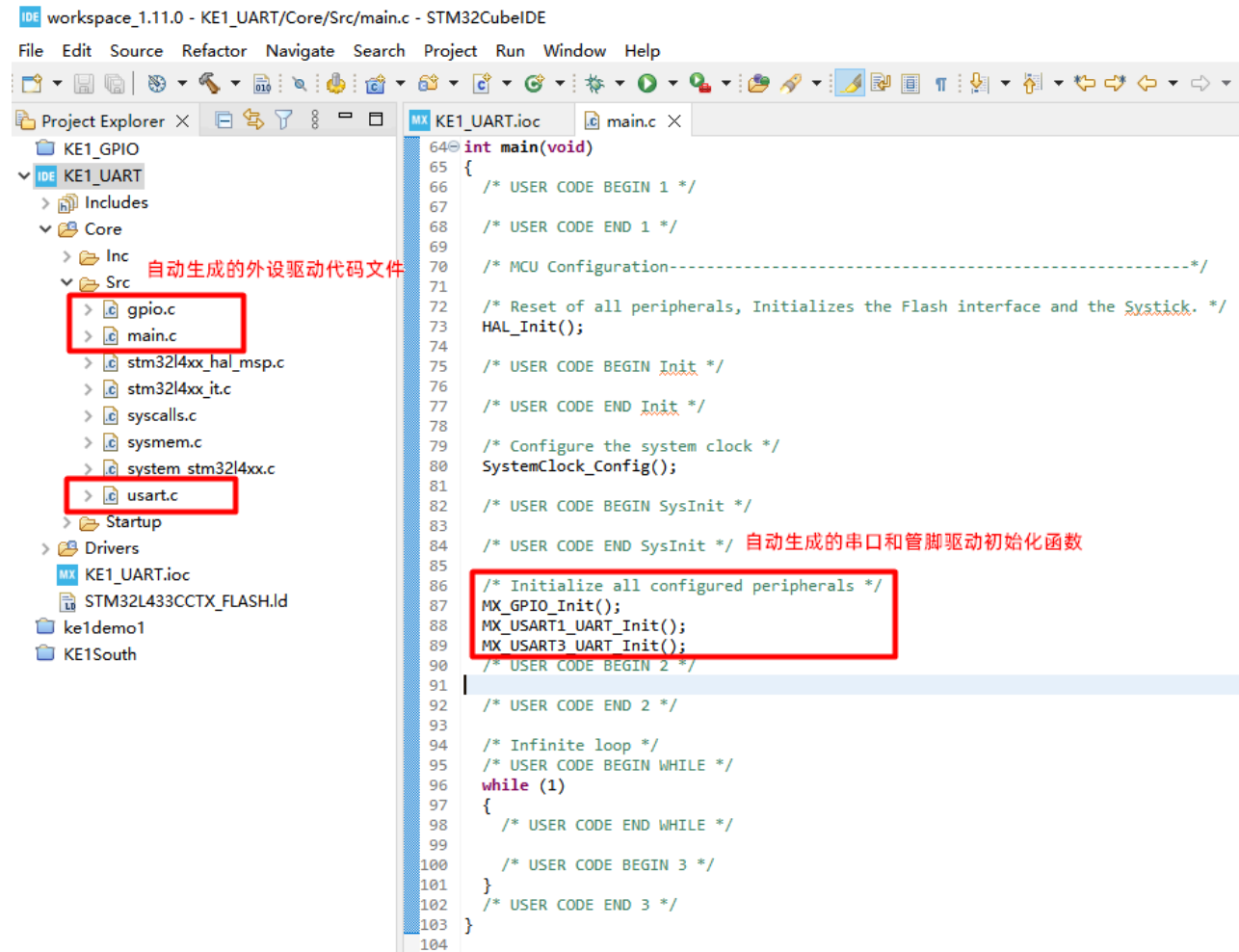
在 **Connectivity** (连接) **USART3** 中，配置工作模式为 **Asynchronous** (异步)，所占用的管脚为 PB10 (USART3\_TX 数据发送) PB11 (USART3\_RX 数据接收) 自动配置。工作波特率为 **9600**。UART3 用来发送 AT 命令控制联网模组 BC28。





### 3. 自动生成驱动代码

外设驱动配置完成即可保存文件，对应的驱动代码将自动生成，并在主函数中自动调用初始化和配置函数。



## UART1 调试串口功能说明

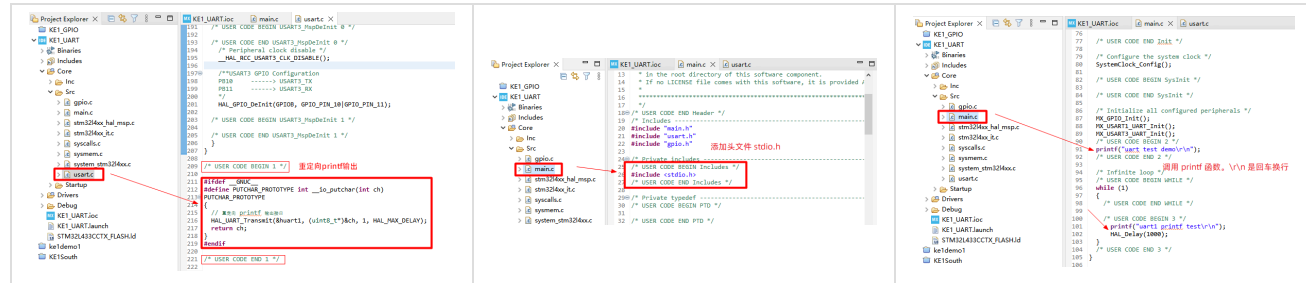
在 C 语言编码中，我们经常使用 printf 作为调试信息输出函数。所以需要重定向 printf 函数从 UART1 输出信息。打开驱动文件 uart.c 文件，添加如下代码

需要使用 printf 时，添加头文件 <stdio.h> 即可。例如本例在文件 main.c 中输出调试信息。

click

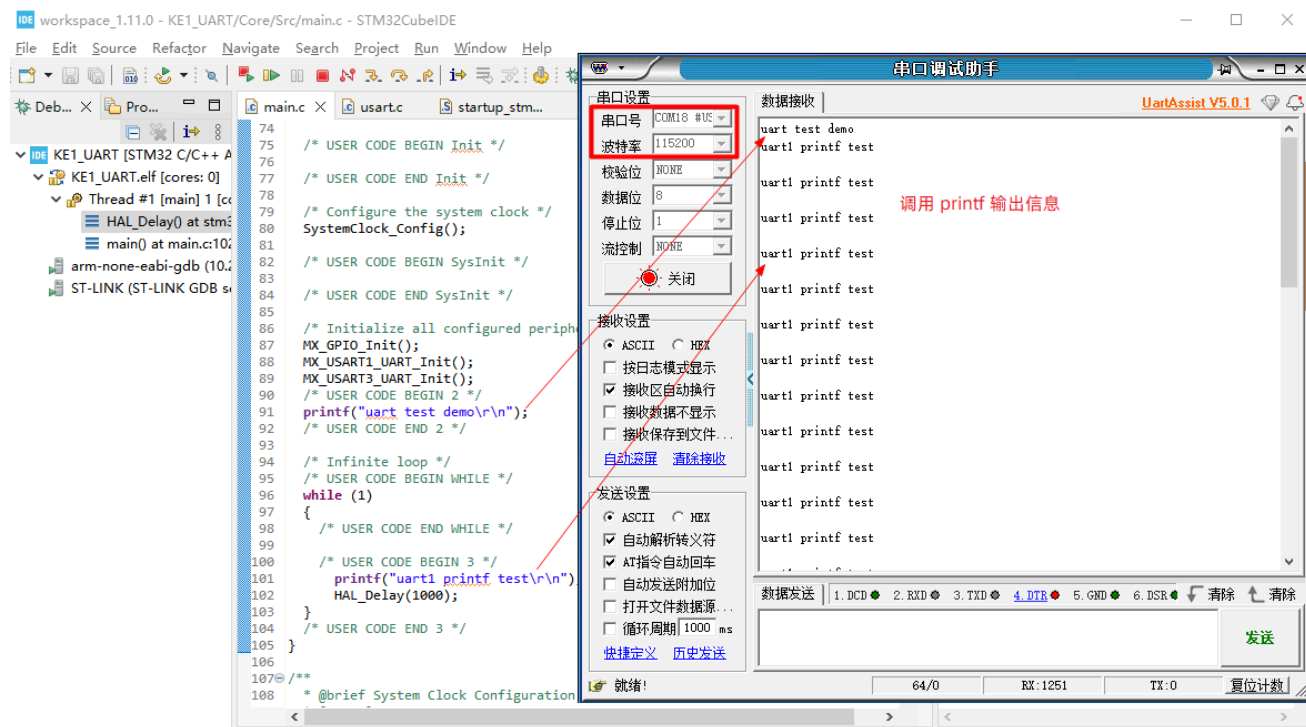
```
#ifdef __GNUC__
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
PUTCHAR_PROTOTYPE
{
```

```
// 重定向 printf 输出接口
HAL_UART_Transmit(&uart1, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
return ch;
}
#endif
```



- 编译并使用ST-Link调试运行代码（请参考【KE1 快速体验】章节）

启动串口工具查看是否能收到代码通过 printf 函数输出的信息。

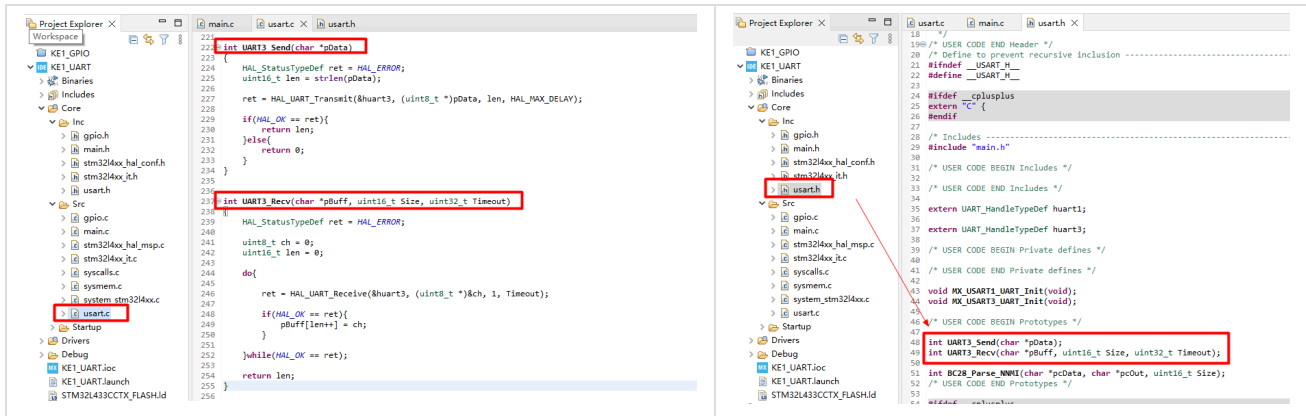


## 串口工具使用帮助

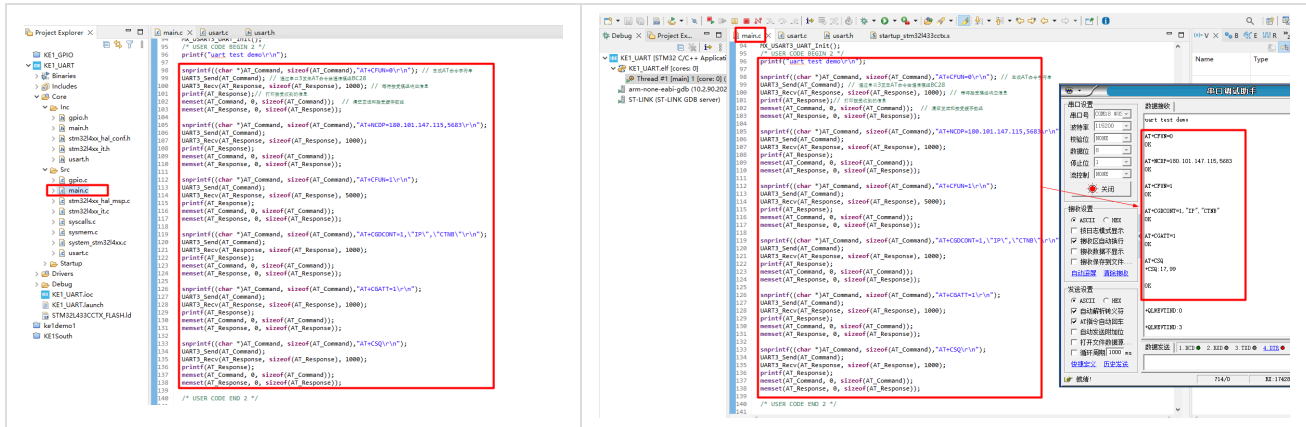
1. 下载串口工具 [UartAssist串口调试助手](#)
2. 启动串口工具，设置串口号为KE1串口号（例如：COMx #USB-SERIAL CH340）
3. 设置串口工具波特率为 115200，其它参数默认即可
4. 点击"打开"按钮连接串口，即可接收KE1发送的数据

## UART3 通讯模组串口功能说明

本例通过使用 UART3 发送 AT 命令控制NB联网模组(BC28)并接收模组的响应数据，实现设备联网功能。为方便使用，重新封装了 UART3 的数据收发函数。函数代码编写在文件 usart.c，声明在 usart.h 中。如下：



接下来就是在 main.c 中调用封装好的 UART3 收发函数，发送 AT命令控制模组联网。详细的AT命令使用请参考【KE1 AT命令操作】章节。编译并调试代码，可在串口工具中查看代码通过 printf 输出的运行信息。



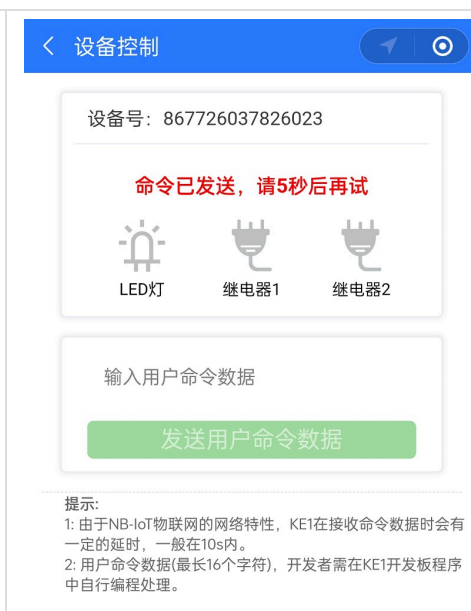
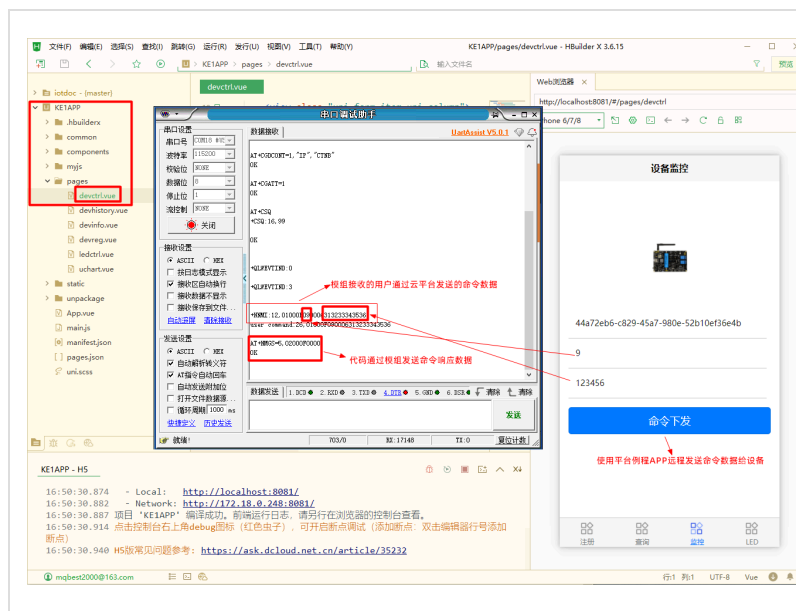
通讯模组成功联网后，用户可通过本平台的例程应用 KE1APP 或者微信小程序下发命令数据。终端代码通过 UART3 循环读取模组接收到的数据，并判断是否为用户通过云平台下发的命令数据。成功处理命令数据后，再通过模组发送命令响应数据到云平台。这样一个最基本的通过串口进行物联网数据通讯案例就完成了。

本例代码仅完成基本工作流程，仅实现了解析模组BC28接收的命令数据函数。例程没有对所有数据或异常进行有效处理，可能有bug，仅供参考。

clike

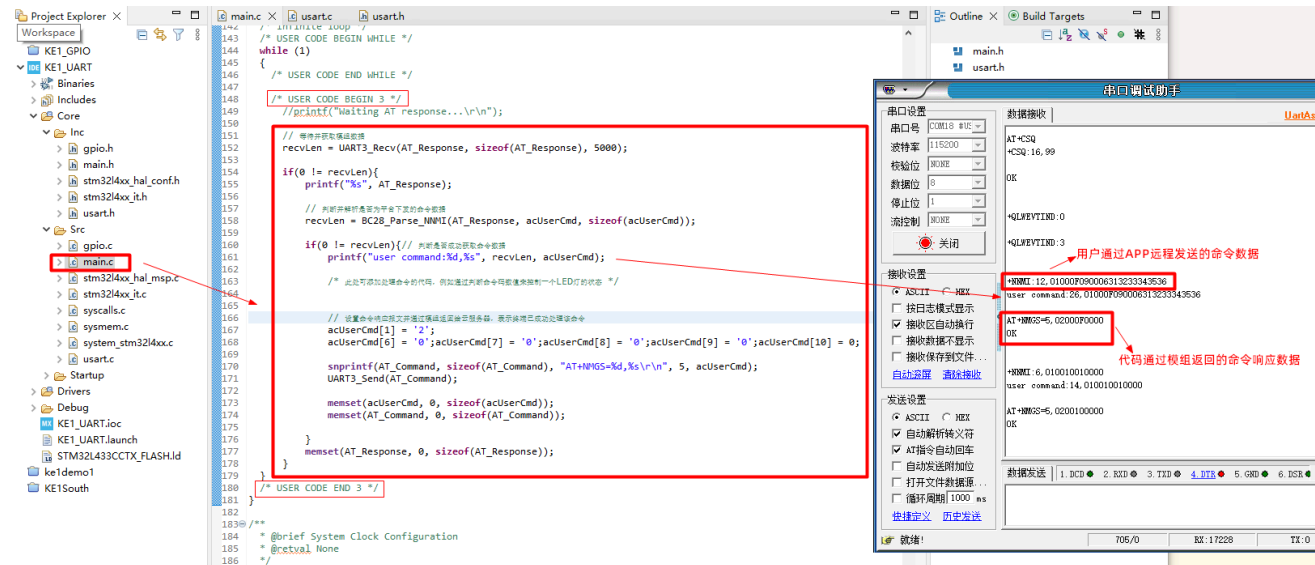
```
//解析模组BC28接收的命令数据函数
int BC28_Parse_NNMI(char *pcData, char *pcOut, uint16_t Size);
```

例程应用 KE1APP 的使用请参考【物联网应用(APP)开发】章节，本平台微信小程序请参考【KE1 快速体验】章节，用户下发的命令数据格式请查看【KE1 AT命令操作】章节



提示:

- 1: 由于NB-IoT物联网的网络特性，KE1在接收命令数据时会有一定的延时，一般在10s内。
- 2: 用户命令数据(最长16个字符)，开发者需在KE1开发板程序中自行编程处理。



## 驱动接口函数说明

本例仅用到串口阻塞式发送和接收两个函数，其它例如使用中断，DMA等收发数据的方式，可自行参考官方例程或 [其它博主介绍](#)

clike

// 使用阻塞方式，发送指定长度的数据，直到发送超时退出。函数输入参数说明请查看驱动源码

HAL\_StatusTypeDef HAL\_UART\_Transmit(UART\_HandleTypeDef \*huart, const uint8\_t \*pData, uint16\_t Size, uint32\_t Timeout)

// 使用阻塞方式，接收指定长度的数据，直到接收超时退出

HAL\_StatusTypeDef HAL\_UART\_Receive(UART\_HandleTypeDef \*huart, uint8\_t \*pData, uint16\_t Size, uint32\_t Timeout);

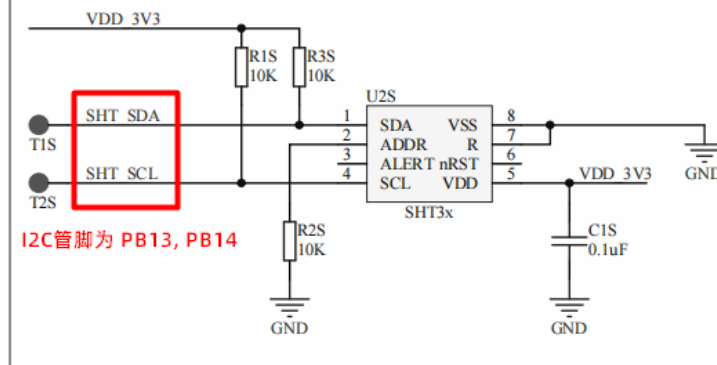
- 下载本例源码 [KE1\\_UART](#)

## I2C 温湿度传感器远程监控

本例将在 KE1\_UART 例程基础上进行开发，所以串口驱动将不再重复说明，仅介绍 I2C 驱动接口的配置和使用。

**本例目标：**通过 I2C 接口读取温湿度传感器(如下图 SHT3x)数据，并通过联网模组将读取的温湿度数据发送到云平台。用户可通过例程APP或小程序查看。

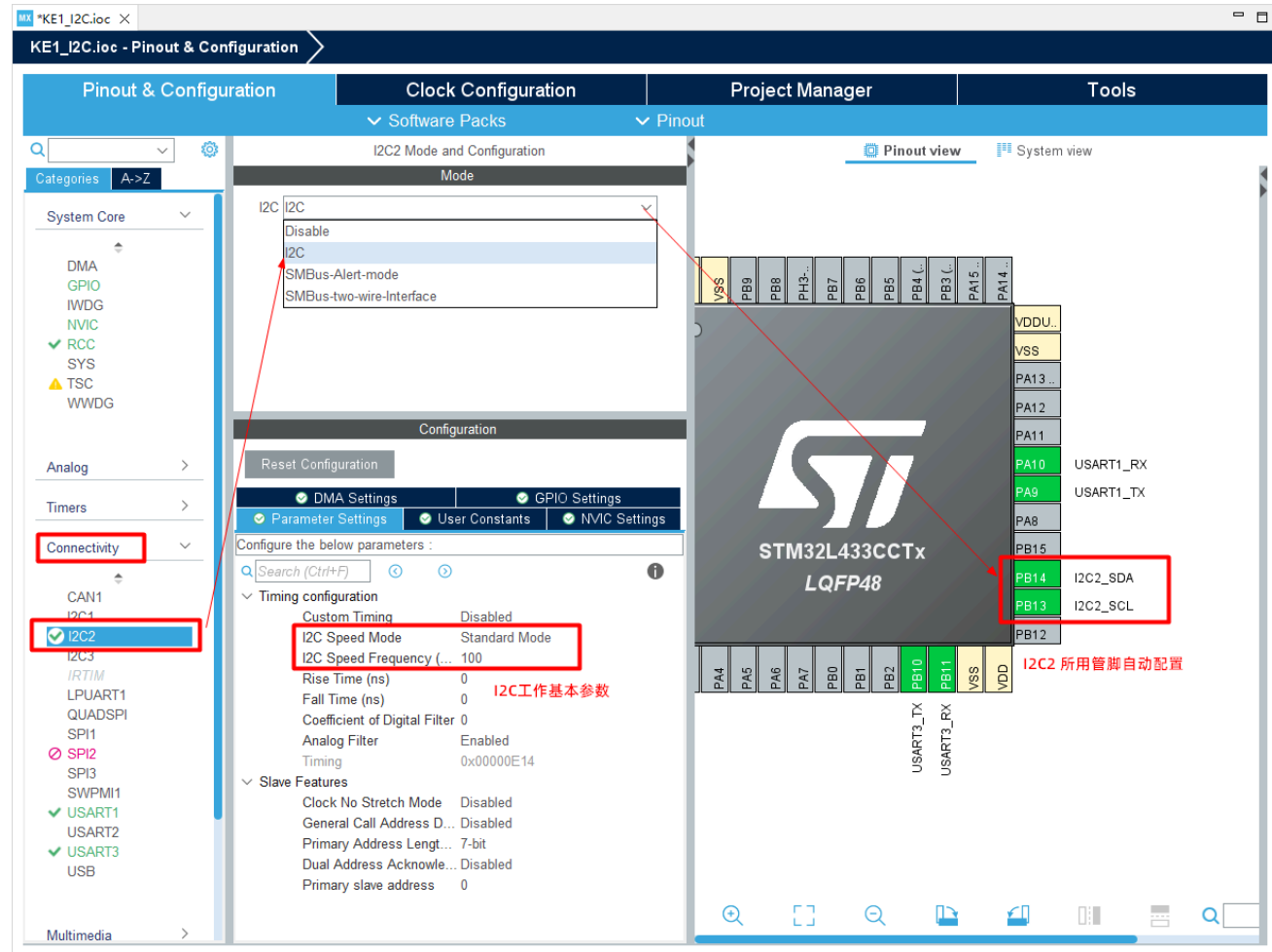
## 温湿度传感器电路



[了解一下 I2C](#)

### 管脚功能配置

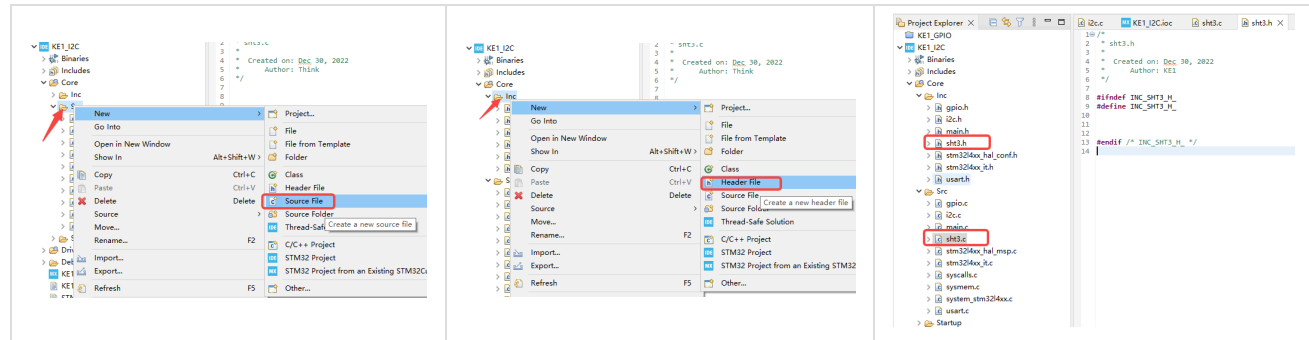
通过电路原理图可知，本例 I2C 使用的管脚为 PB13(SCL)，PB14(SDA)，为第 2 组 I2C 通道。所以需在 **Connectivity** (连接) **I2C2** (I2C通信接口2) 中使能 I2C 功能。可设置工作速率 (I2C Speed Frequency) 20 KHz。设置完成保持配置，自动生成驱动代码。



## 功能代码说明

添加新的用户源文件（sht3.c和sht3.h），通过 I2C 接口实现读取温湿度传感器(SHT3x)的驱动代码





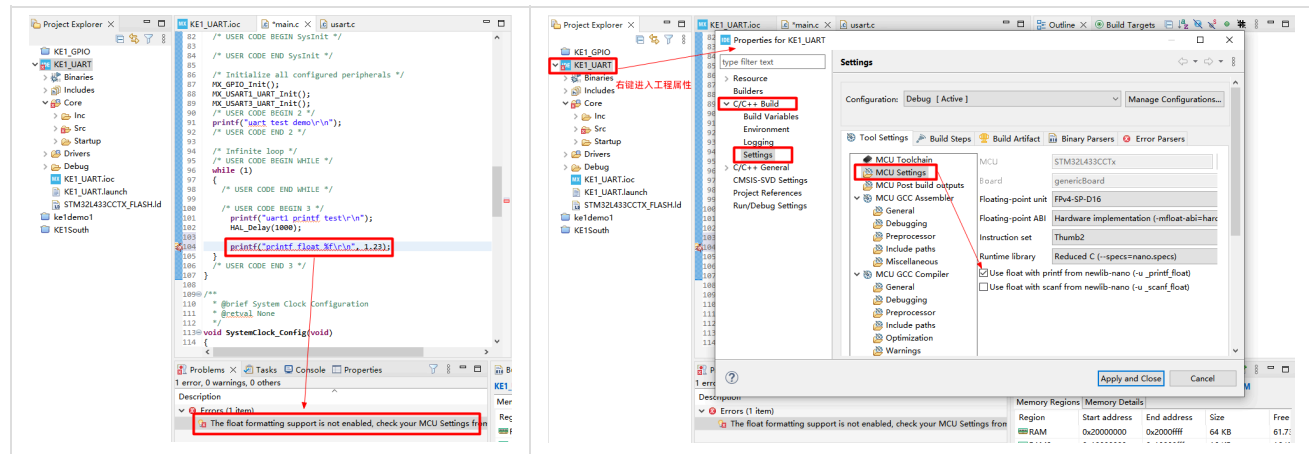
SHT3x 驱动代码参考 [网络资料](#)

本例主流程代码如下：

- 下载本例源码 [KE1\\_I2C](#)

## 扩展知识

1. 函数 printf 默认不支持浮点数(float)格式化输出，需要勾选使能支持浮点数，否则会报错！



## 物联网应用(APP)开发

### APP 快速开发

本章节将通过 [uniapp](#) 框架，并结合本平台提供的接口(URL)，快速实现对KE1进行远程监控的应用APP。可以在此基础上充分发挥自己的想象力，实现更多有趣的功能，以此更加深刻的体会物联网的价值。

之所以选择使用 uniapp 框架进行APP开发，是因为它支持多平台，入手简单！

## 安装开发工具

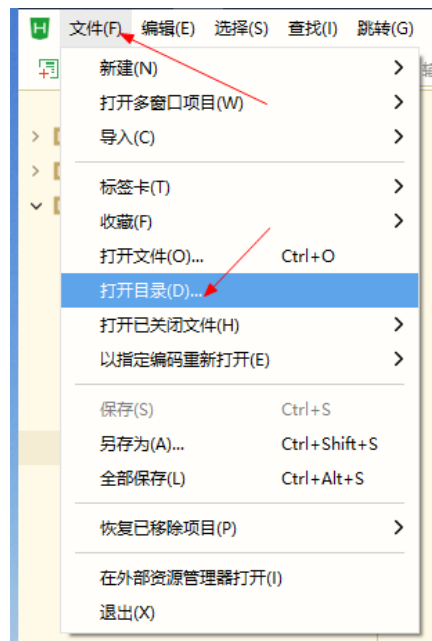
uniapp 开发工具是官方提供的 [HBuilder X](#) 安装详情参考官方 [安装说明](#)

## 获取例程APP

本平台提供了一套基于 uniapp 框架的应用APP源码（KE1APP），演示了如何通过本平台提供的接口与 KE1 终端设备进行数据交互的方法。[点击下载源码](#)

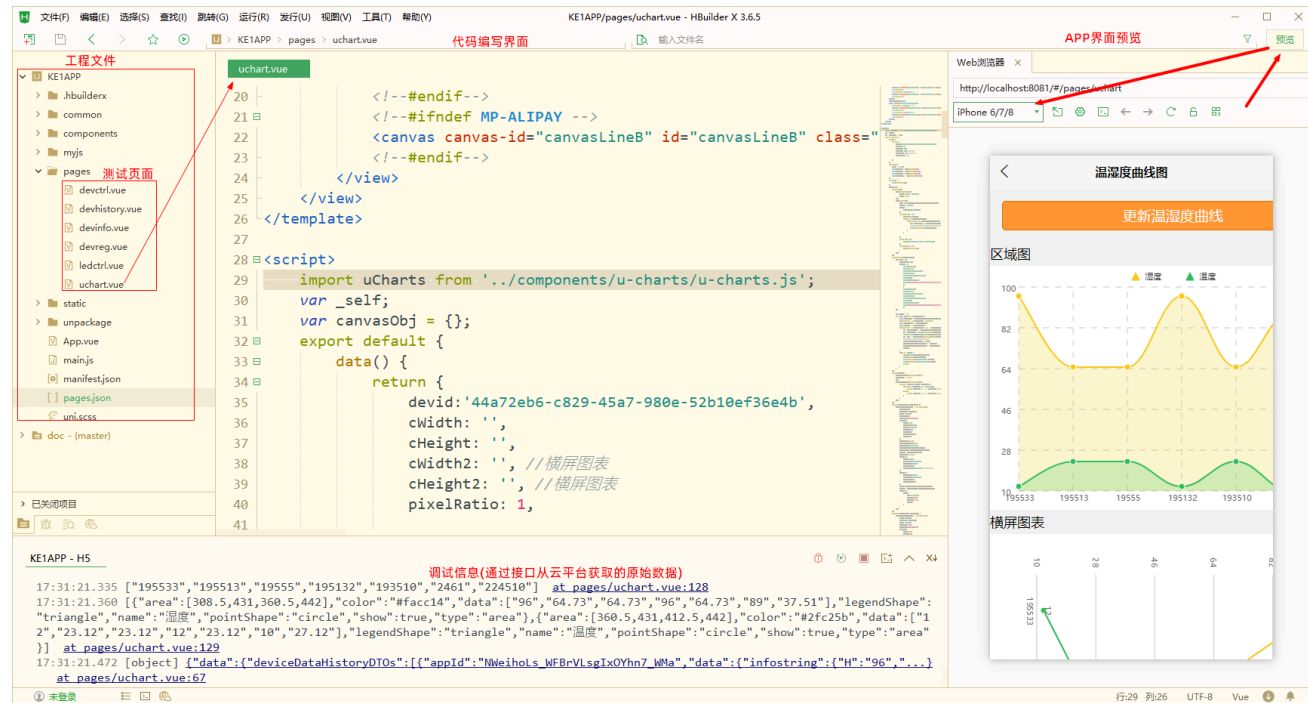
### 1. 导入例程源码

如下图，启动开发工具，直接打开解压出来的KE1APP源码根目录即可



### 2. 工具界面简介

uniapp 工程的详细介绍请参考 [官方文档](#)，写很清楚，这里就不啰嗦了

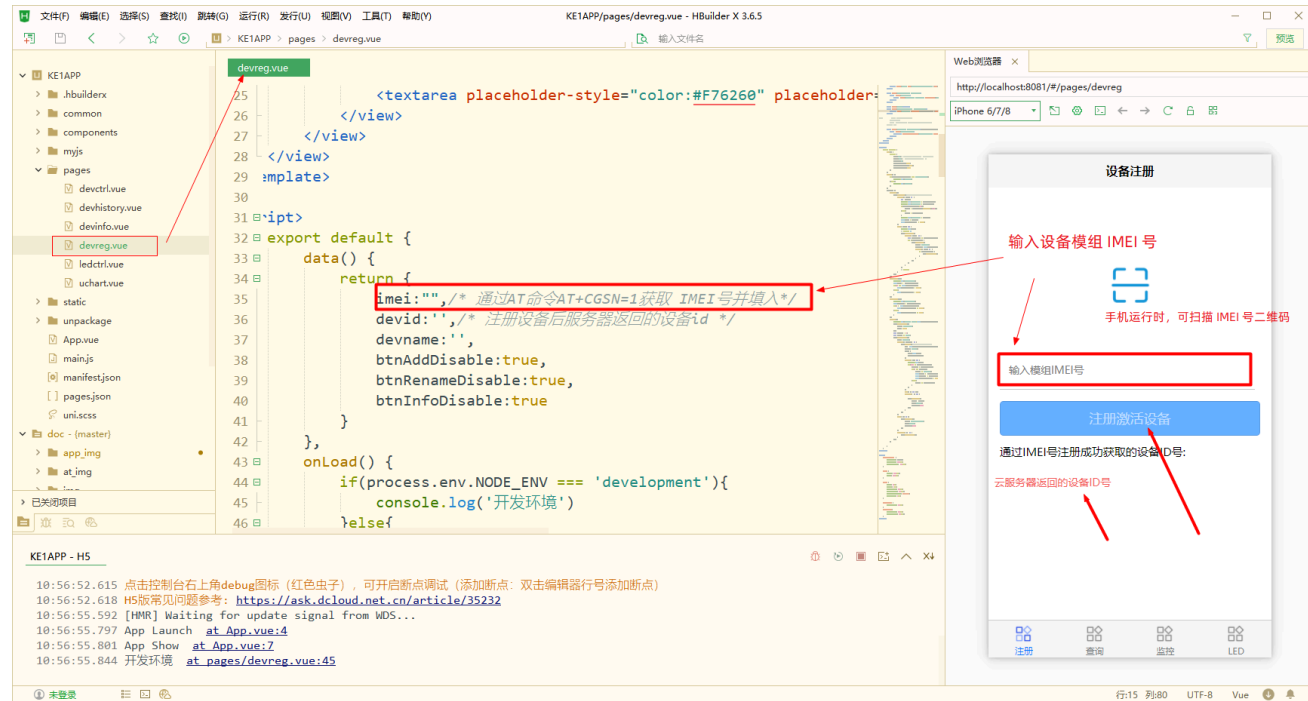


## APP界面源码文件介绍

### 设备模组注册激活演示界面 devreg.vue

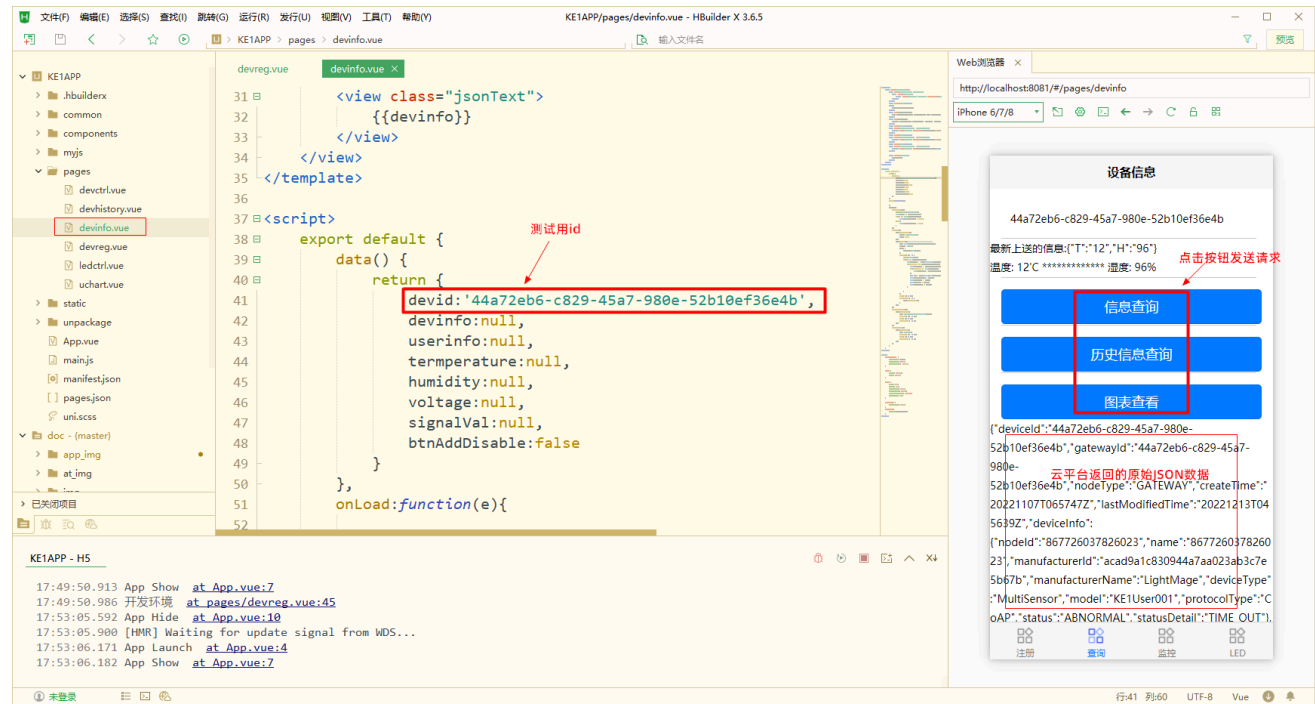
在源码红框中填入通信模组IMEI号。启动APP预览界面(查看方法)，点击注册激活按钮发送请求，成功后云平台会返回设备ID（devid）。保存好这个ID，它将用在其它几个界面中。

获取IMEI号请参考【KE1 AT 命令操作入门】章节，或者直接从模组上读取



### 终端数据查询功能演示界面 devinfo.vue

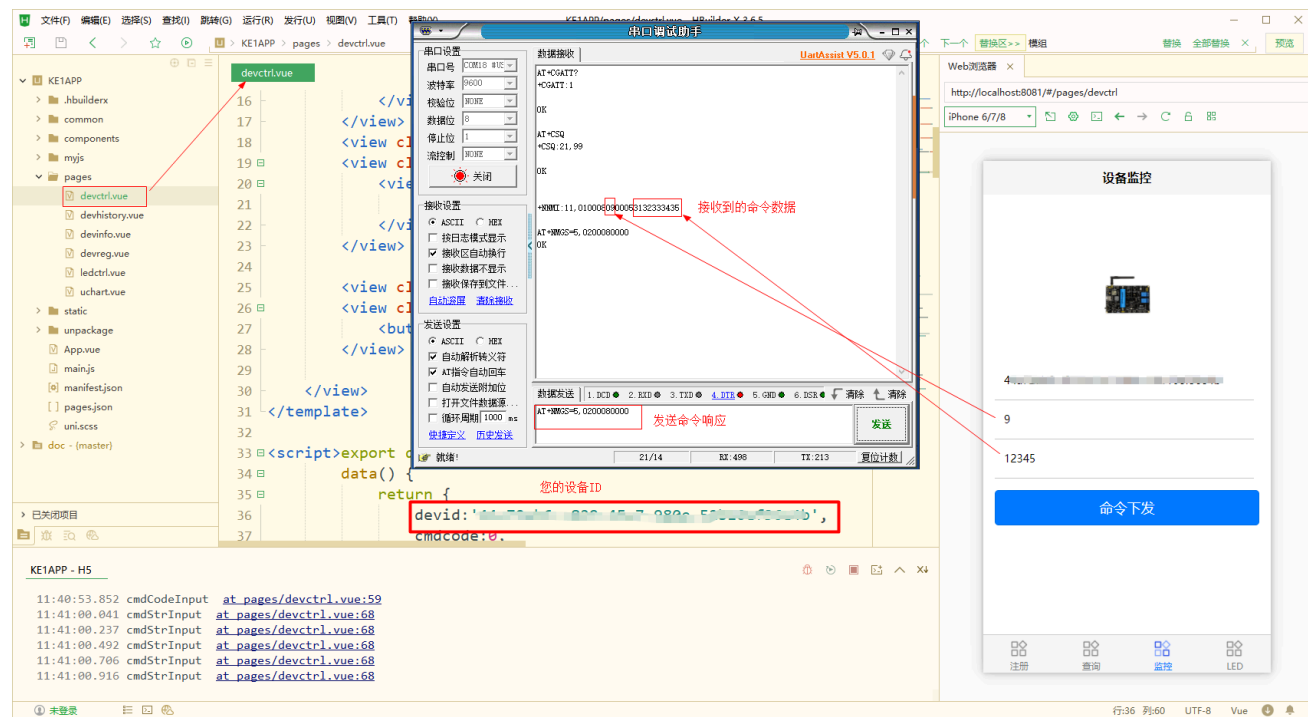
在源码红框中填入自己的设备ID (devid)。例程中默认的devid为测试设备id, 虽然可以直接使用, 但是获取的是样例设备数据, 而不是自己的设备数据。



## 终端控制（命令下发）功能演示界面 devctrl.vue

在源码红框中填入自己的设备ID（devid），并在预览界面输入自定义命令码和参数，然后点击命令下发。

在测试该功能时，您的KE1通信模组需要已经成功联网，可参考【KE1 AT命令操作入门】章节手动联网，或者参考【KE1 快速体验】章节运行综合例程进行联网，并通过串口工具查看模组收到的命令数据。下图案例是通过AT命令手动联网进行测试。



## 其他界面文件说明

- devhistory.vue: 终端历史信息展示界面
- ledctrl.vue: 模拟控制LED
- uchart.vue: 通过图表控件显示数据

## 安装应用到手机

请参考官方文档 [真机运行](#)

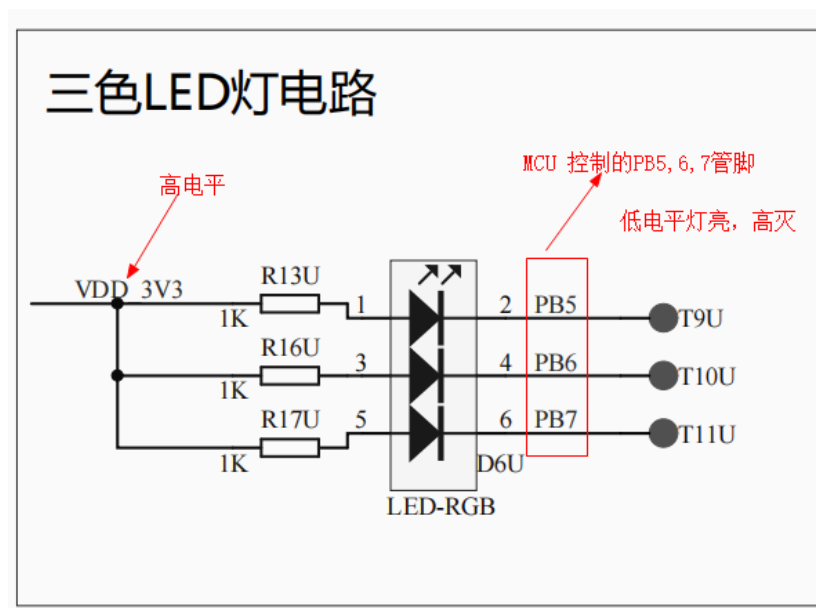
# STM32CubeIDE 使用简介

## STM32CubeIDE 使用简介

本章将通过一个在单片机开发中，最经典也是最基础的案例：[跑马灯](#)，来简单描述一下 STM32CubeIDE (v1.10.1) 的基础使用方法和注意事项，更多其它功能将在【物联网终端(STM32)开发】章节中通过不同的案例进行演示说明，当然您也可以参考[其它博主帮助文档](#)学习 STM32CubeIDE

的使用 (可能有差异, 但大同小异)。理论与实践相结合, 让学习更轻松!

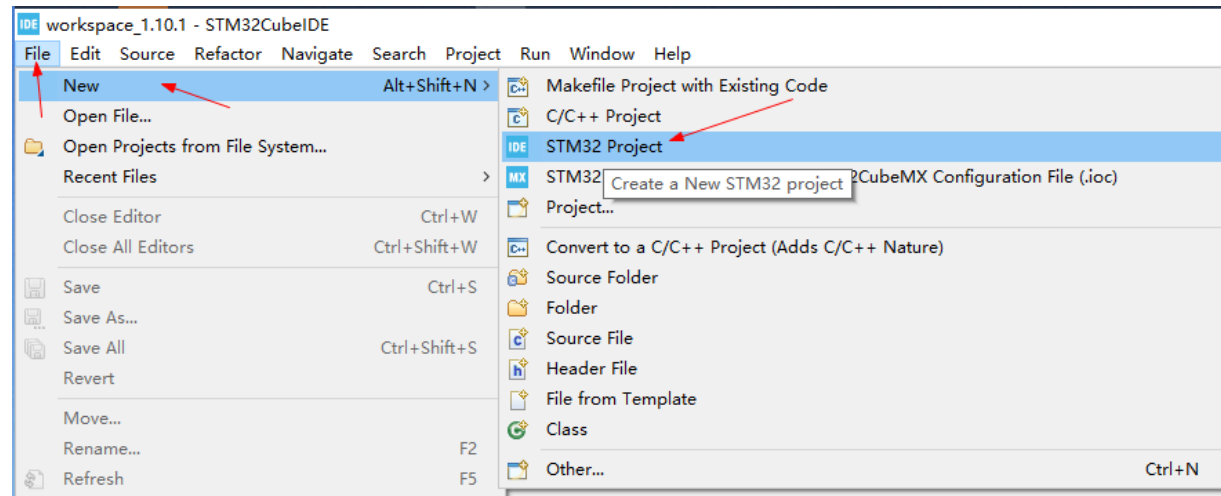
不过, 在开始之前, 我们需要有一定的 C 语言和数模电基础, 能看懂基础的电路原理图。如下图, 就是本例需要了解的三个LED灯控制电路图。



好了, 接下开始我们的第一个项目: 跑马灯! (控制三个LED灯交替闪烁) 如上图所示, 本例需要控制的GPIO管脚为 PB5, PB6, PB7。低电平灯亮, 高电平灯灭。

电路原理图请参考 "小熊座KE1开发板原理图.pdf", 该文件在[KE1综合实例源码](#) *KE1South* 源码包中

## 新建项目



如果弹出软件更新下载框，请等待其自动下载完即可。然后再次操作

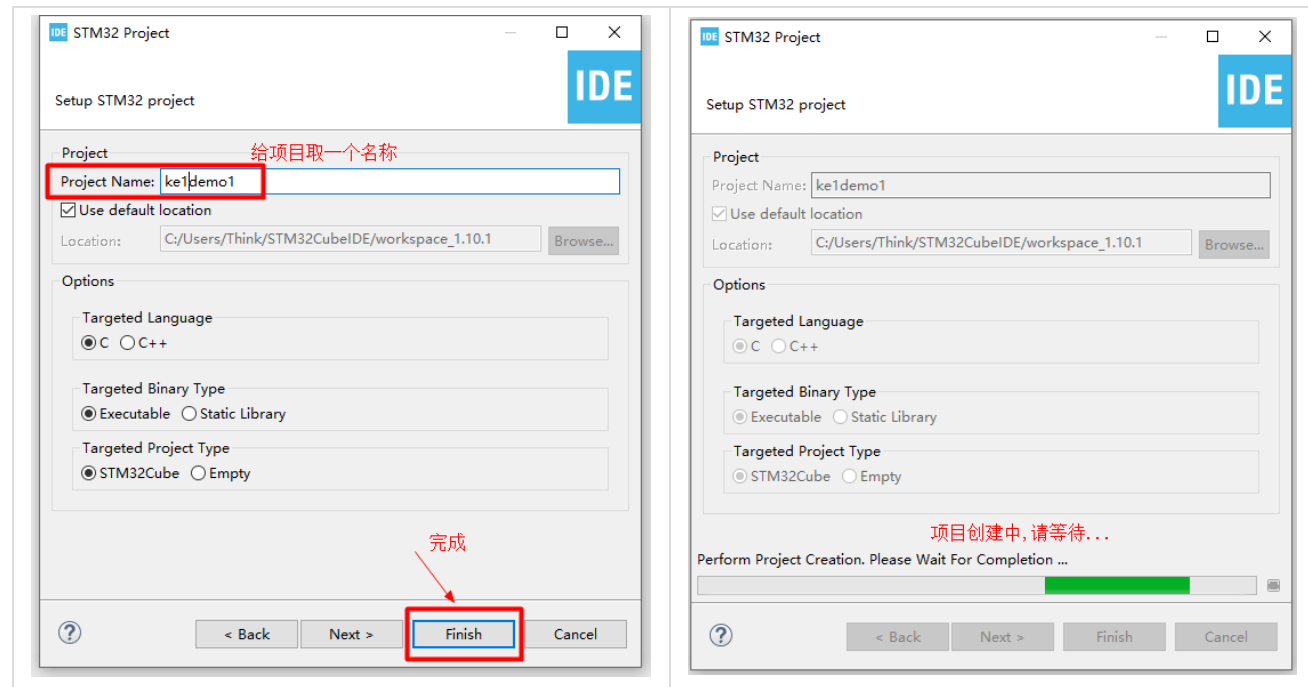
## 选择目标芯片

在 **Commercial Part Number** 框中输入目标芯片型号进行检索，选中具体芯片，然后点"Next"

KE1使用的芯片型号为: STM32L433CCT6（可能有不同，具体请查看KE1板载芯片上所标识的型号）

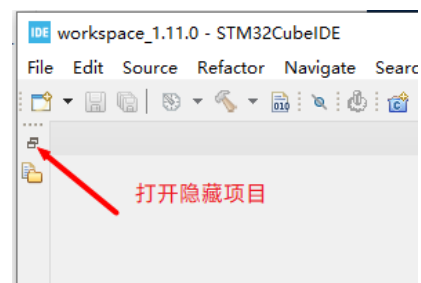






## 新建项目成功

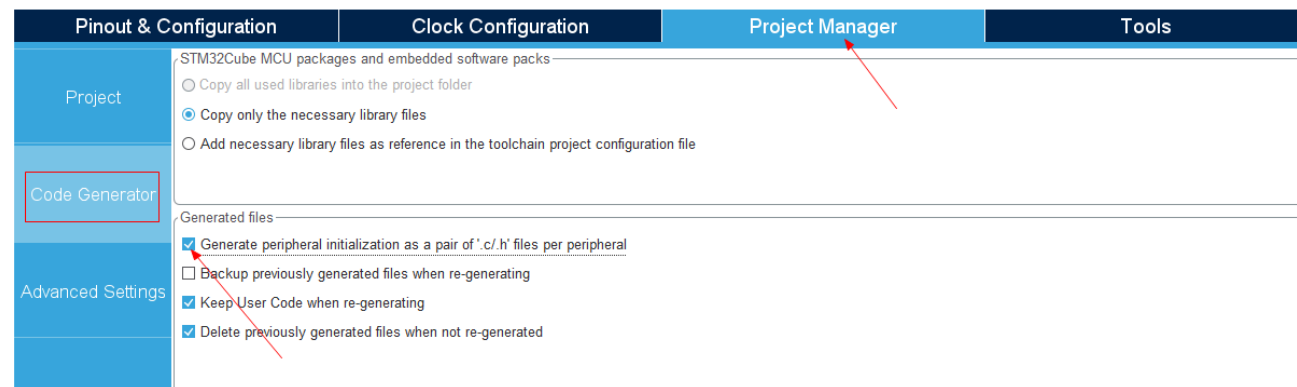
项目新建成功，打开项目工作界面如下





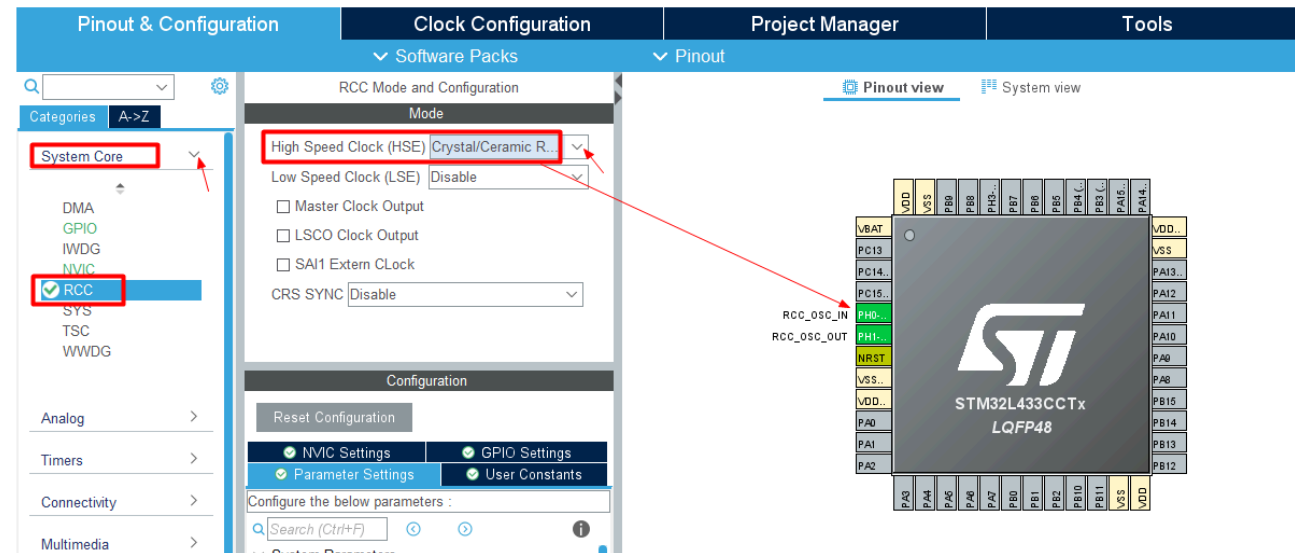
## 配置项目

在 **Project Manager** (项目管理) **Code Generator** (代码生成) 中, 勾选使能自动生成外设驱动的 .c 和 .h 文件



## 配置芯片工作晶振

在 **System core** (系统核心) **RCC** (复位和时钟配置)中配置HSE工作晶振为 **Crystal/Ceramic Resonator** (外部晶振)，对应管脚将自动被配置。

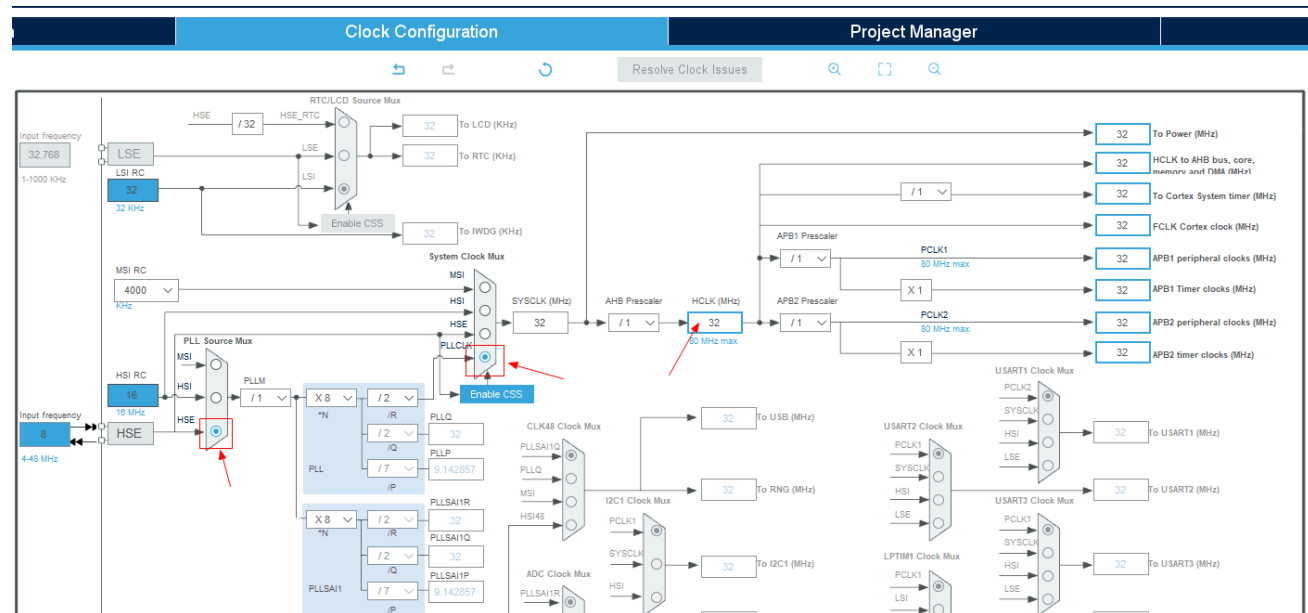


## 配置所需GPIO管脚属性

在右侧芯片管脚示意图上，分别点击本例所需管脚 PB5, PB6, PB7，在弹出的列表中选择工作模式为 **GPIO\_Output** (输出模式)。

并在**System core** (系统核心)**GPIO** (通用输入/输出接口)中配置所有管脚的默认输出为 **High**(高电平) (即对应的LED灯灭，取决于电路原理图)。



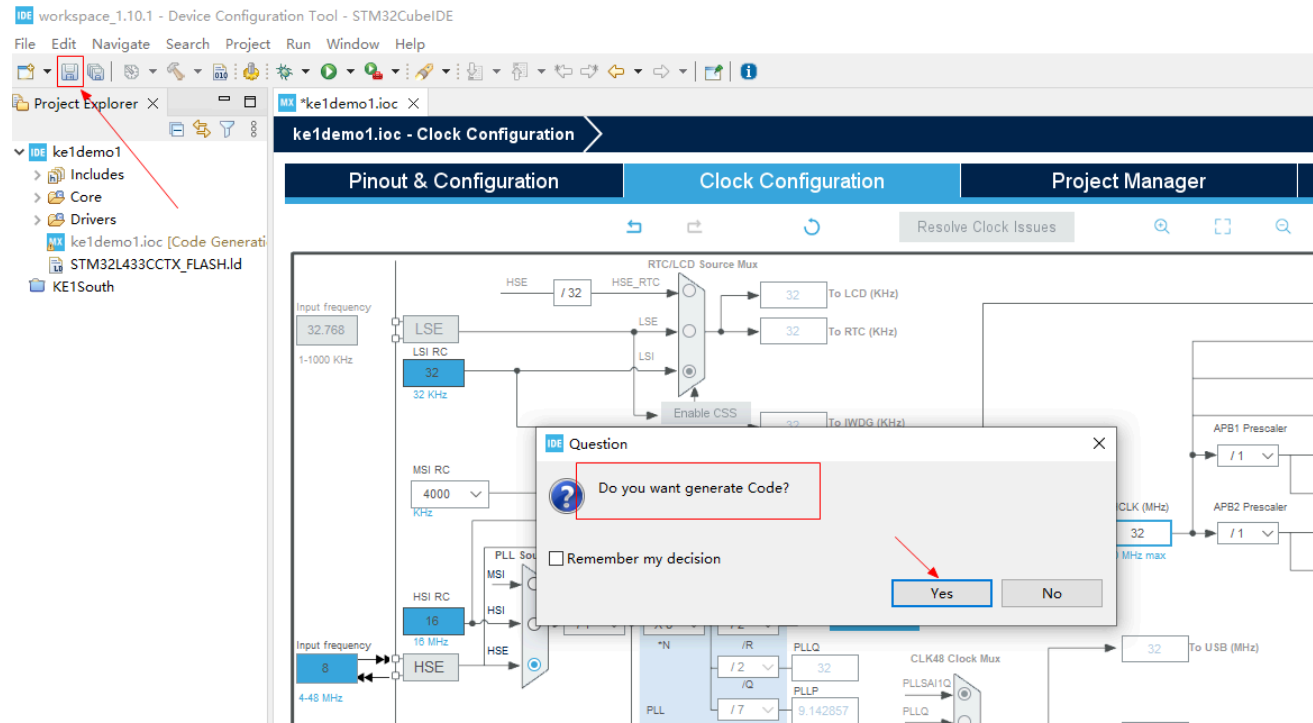


STM32时钟树中有6种时钟源，如下所示：

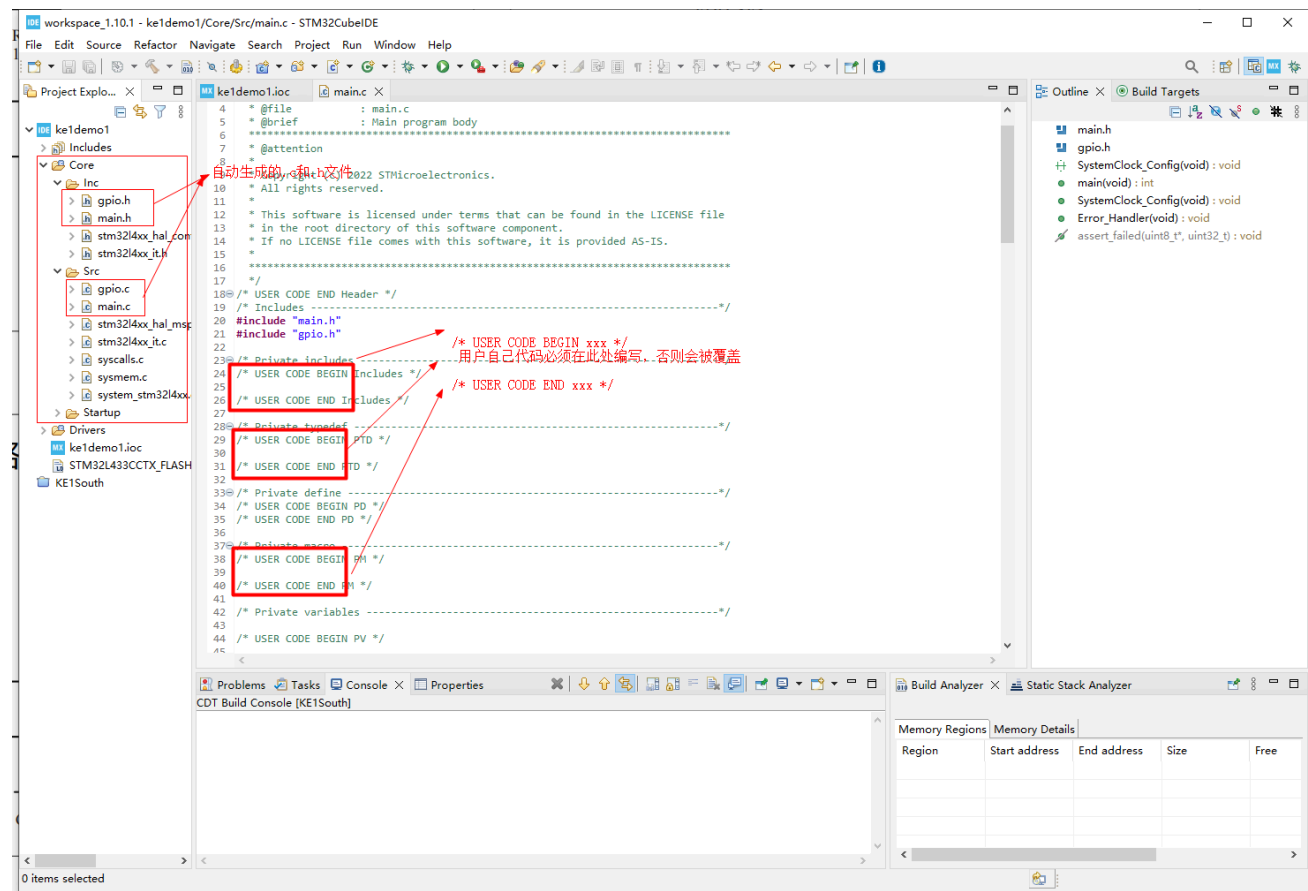
- HSI 是高速内部时钟，RC振荡器，频率为8MHz
- HSE 是高速外部时钟，可接石英/陶瓷谐振器，或者接外部时钟源，频率范围为4MHz~16MHz
- LSI 是低速内部时钟，RC振荡器，频率为40KHz
- LSE 是低速外部时钟，接频率为32.768KHz的石英晶体
- PLL 为锁相环倍频输出

## 自动生成驱动代码

配置无误后保存工程配置文件(.ioc)，将提示自动生成代码。点击确认，等待完成即可。



查看自动生成的代码内容。代码中有很多如图类似的注释，从其字面意思即可理解，用户代码只能编写在该类似注释区域内！如果您写在其它位置，那么一旦重新配置了工程，自动生成代码时会覆盖您的代码。切记！！



click

```
/* USER CODE BEGIN xxx */
```

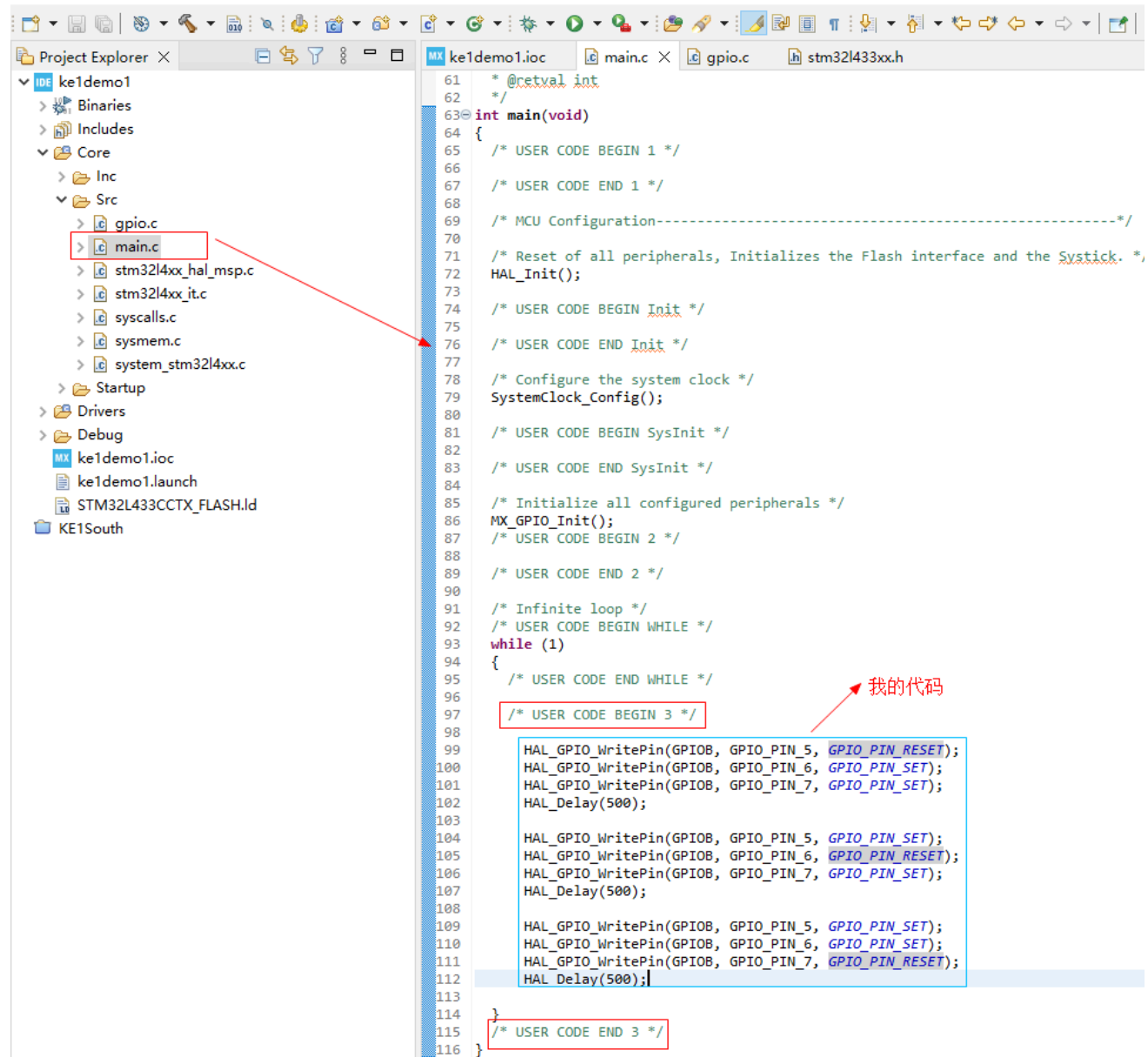
//您的代码只能写在该位置，其它地方会被覆盖！切记！切记！

```
/* USER CODE END xxx */
```

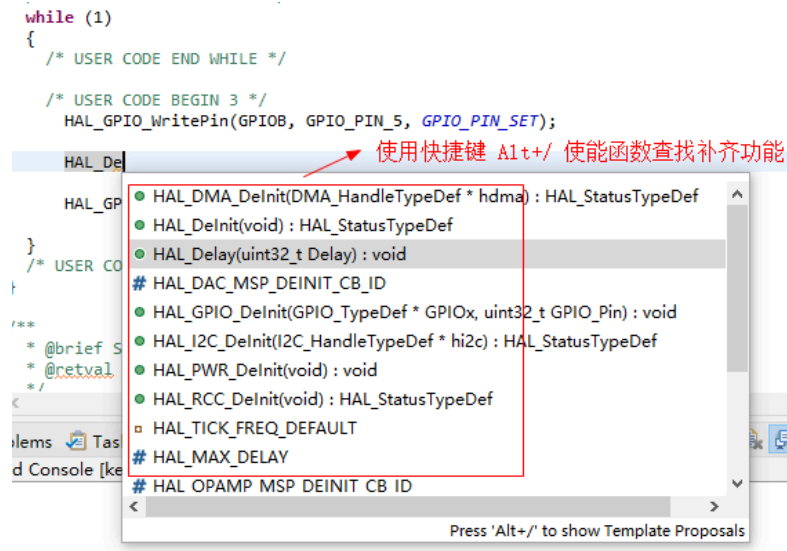
## 编写功能代码

打开自动生成的main.c文件，将自己的代码编写在正确的区域





可使用快捷键 **Alt+/** 自动补齐函数，提高编程效率



本例 GPIO 操作函数说明

```
// 设置GPIOB组中的 GPIO_PIN_5 第5管脚为 GPIO_PIN_SET 输出高电平
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);

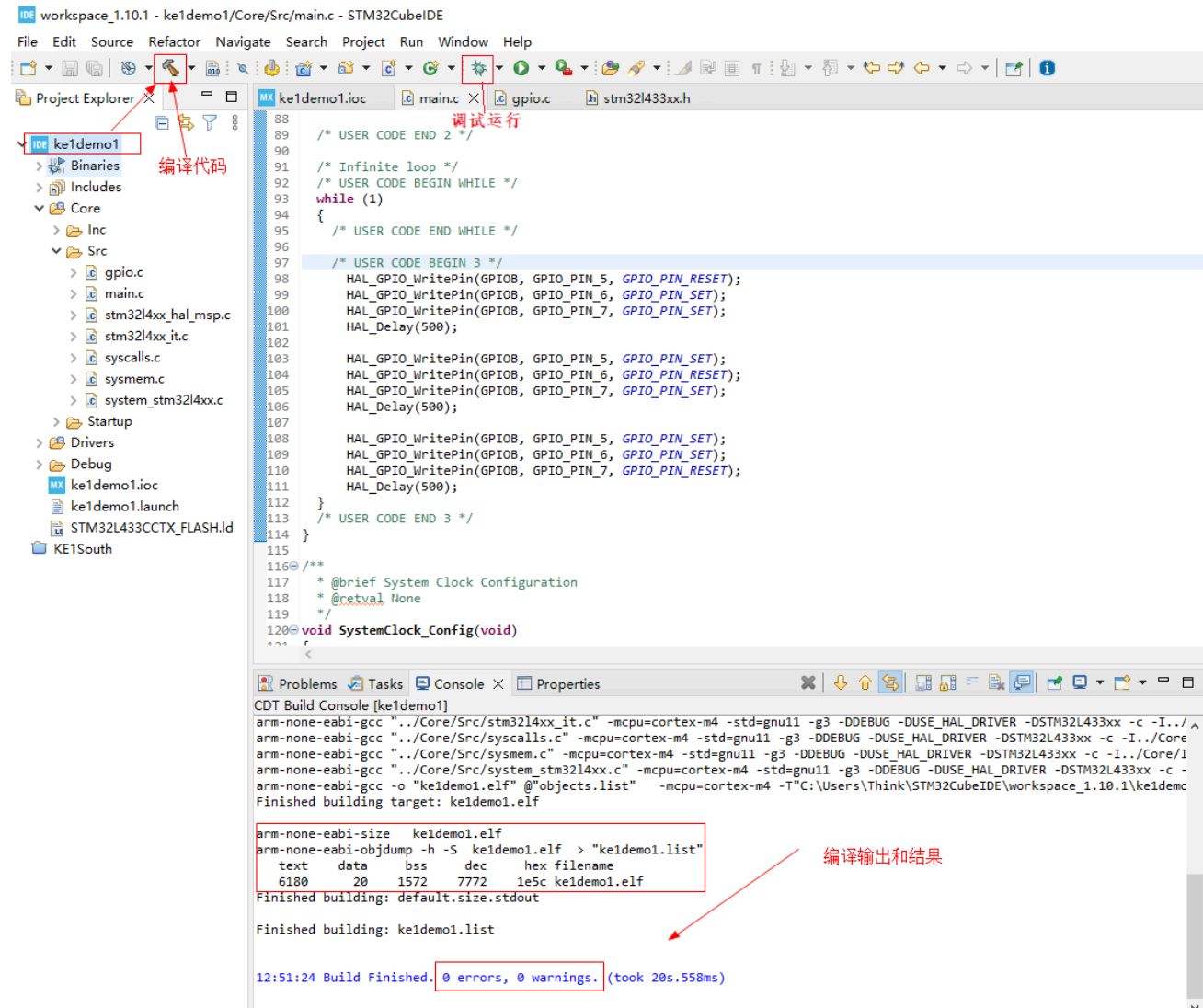
// 设置GPIOB组中的 GPIO_PIN_6 第6管脚为 GPIO_PIN_RESET 输出低电平
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);

//延时500ms, 该函数为 STM32 HAL库函数中自带的延时函数（毫秒级）
HAL_Delay(500);
```

## 编译并调试代码

代码编写完记得及时保存文件。选中工程，开始编译代码，直到没有错误提示。然后就可以调试代码了，如何调试运行请参考【KE1 快速体验】章节。

click

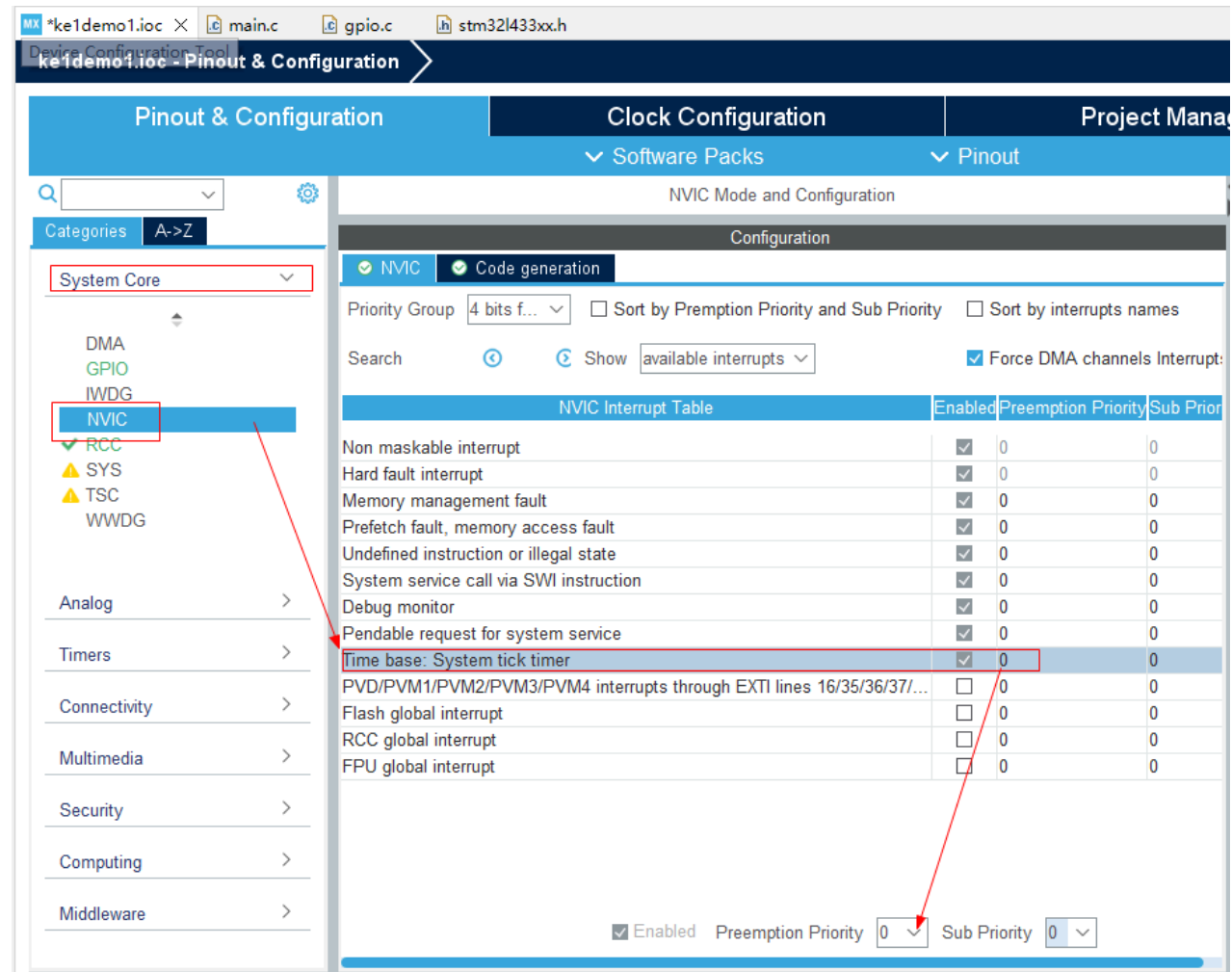




### 本例可能异常说明

如果代码没有按照逻辑控制LED灯闪烁，那么可能是函数 `HAL_Delay()` 失效。

解决办法就是在 **System core** (系统核心) **NVIC** (中断控制器)中修改 **Time base: System tick timer** (系统定时器) 的中断优先级从默认值 15 改成 0 (数值越小，优先级越高)，0 为最高优先级。



设置系统定时器中断为最高优先级，使函数"SysTick\_Handler"正常工作，其代码如下。

```
// 在 stm32l4xx_it.c 中
void SysTick_Handler(void)
{
    /* USER CODE BEGIN SysTick_IRQn 0 */

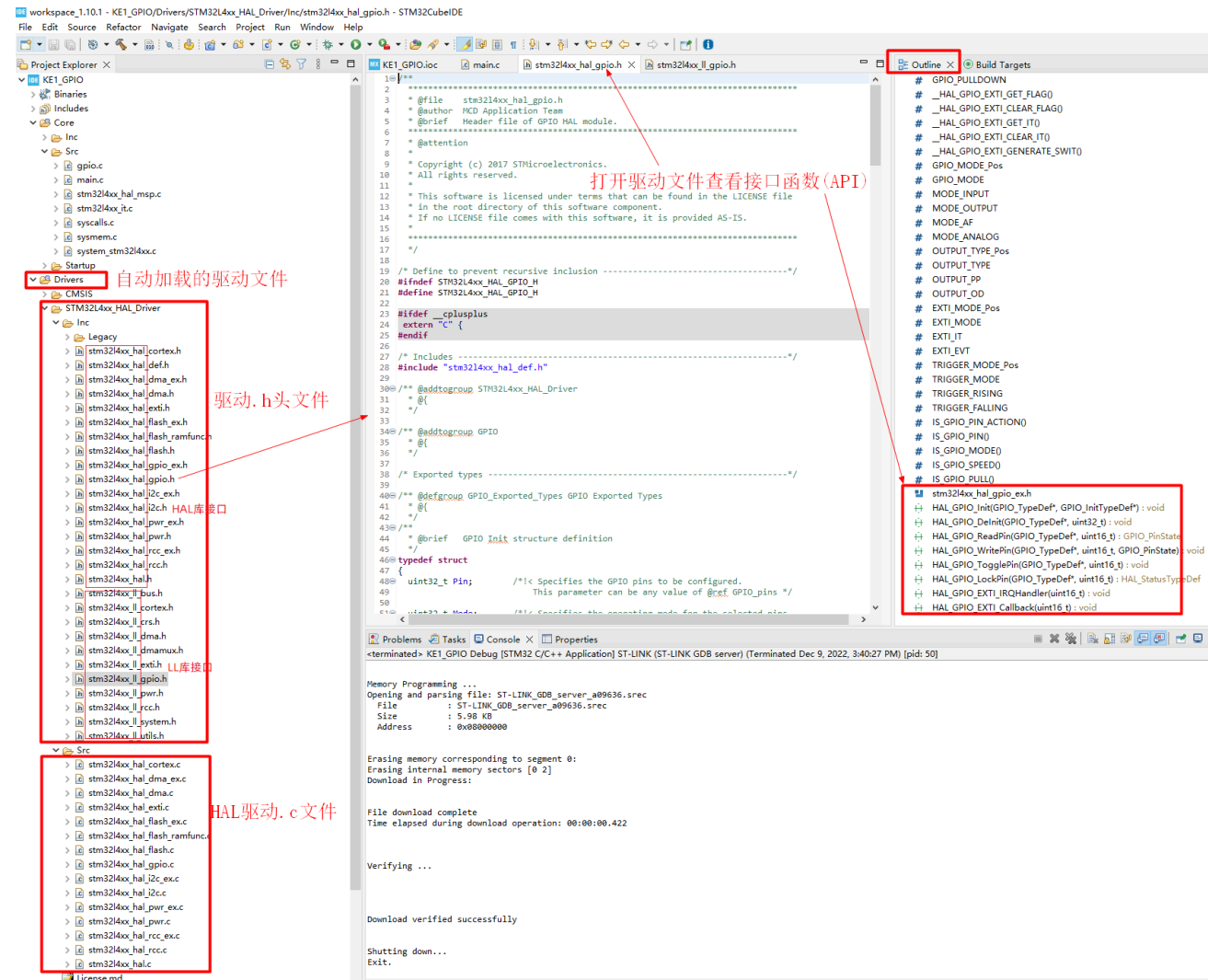
    /* USER CODE END SysTick_IRQn 0 */
    HAL_IncTick();
    /* USER CODE BEGIN SysTick_IRQn 1 */
```

click

```
/* USER CODE END SysTick_IRQn 1 */  
}
```

## 如何查看官方驱动接口(API)

例程配置完所需外设驱动后，保持配置时，所需驱动源码将自动加载到项目中，如下图所示。本例仅介绍如何获取GPIO驱动接口函数，但其它驱动接口函数的查询是类似的，此处仅给一个举一反三的例子。

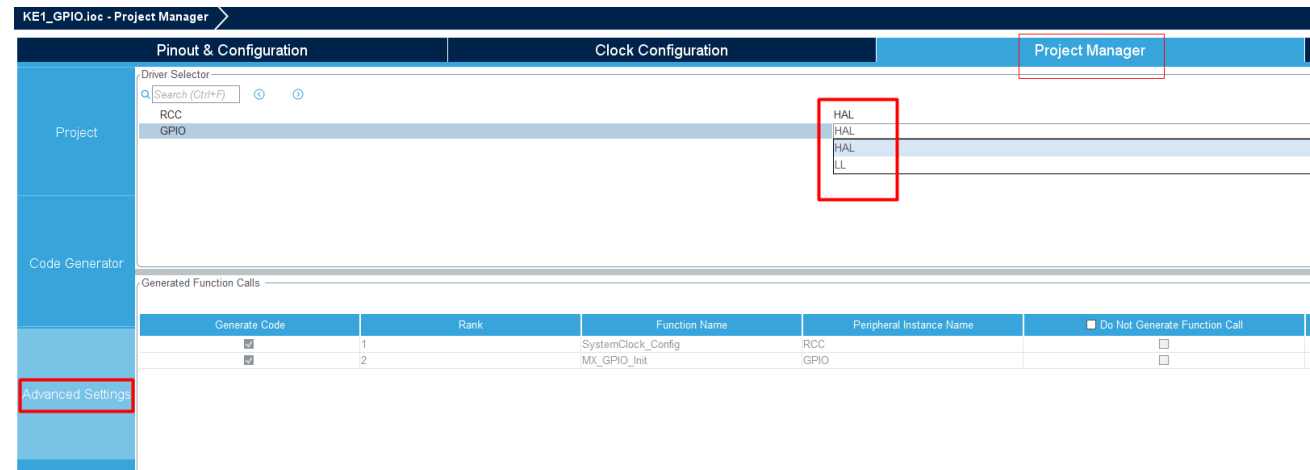


## 如何选择驱动库函数类型

ST提供了两种库函数：LL库和HAL库

- LL 库是一个精简的库，更底层，编译出来的代码更小，执行更快，但是因为更底层，所以使用上比较难，在对代码有较高要求时可选用。
- HAL 库是硬件抽象层库 [点击了解](#)，默认库函数，使用起来简单直观，容易入手学习（**推荐**）。

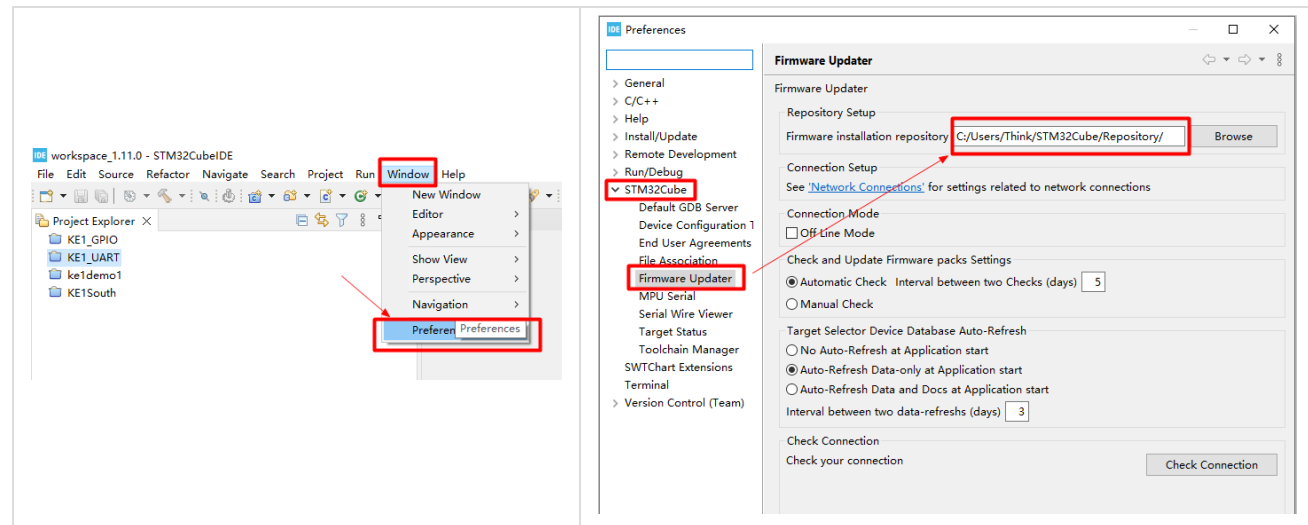
在 **Project Manager** (项目管理) **Advanced Settings** (高级设置) 中可对各个驱动进行库类型选择



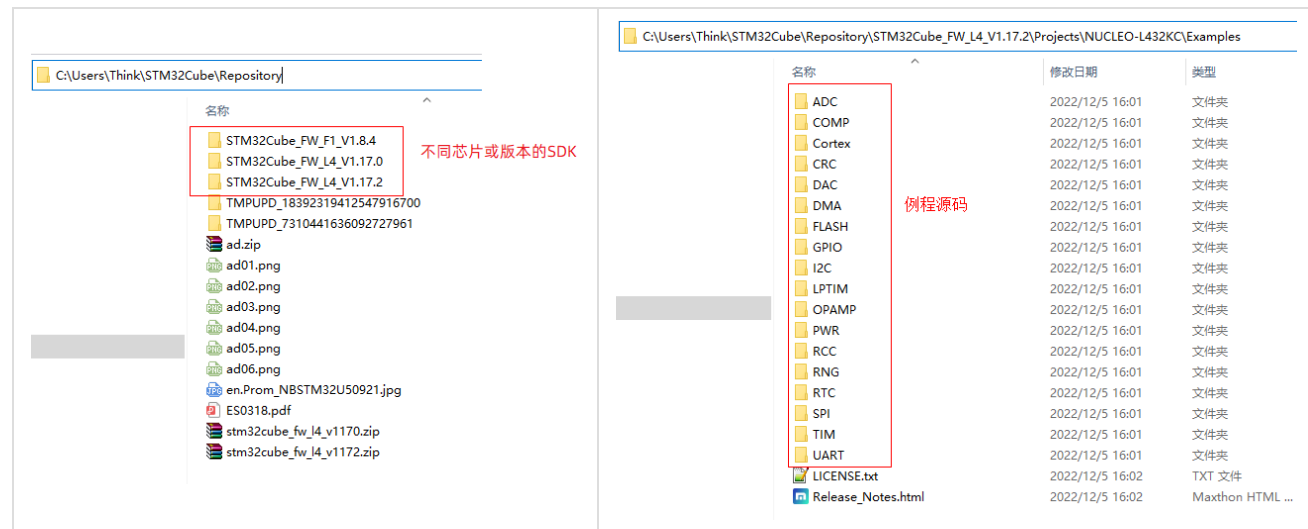
## 了解官方固件驱动源码

STM32CubeIDE 会自动下载对应芯片的驱动固件(SDK)源码包，并保存在本地的仓库目录。各SDK里面有对应芯片的例程源码和接口函数使用手册。通过学习官方例程和手册，可以快速掌握 STM32 各类外设驱动的使用方法。

[查看固件仓库目录](#)



## 查看驱动例程

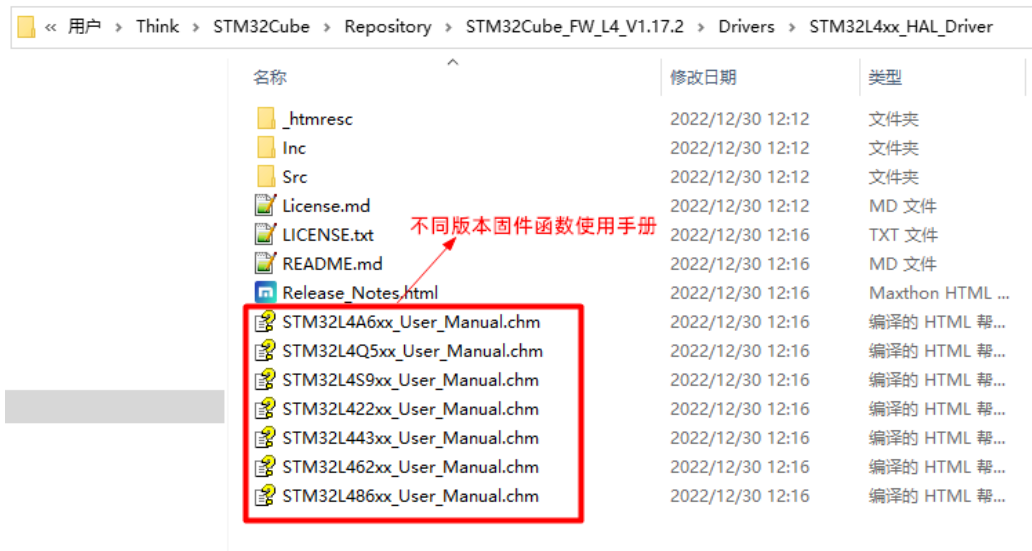


## 查看驱动接口函数使用手册

进入固件仓库目录，在对应的驱动目录中可查看其使用手册。

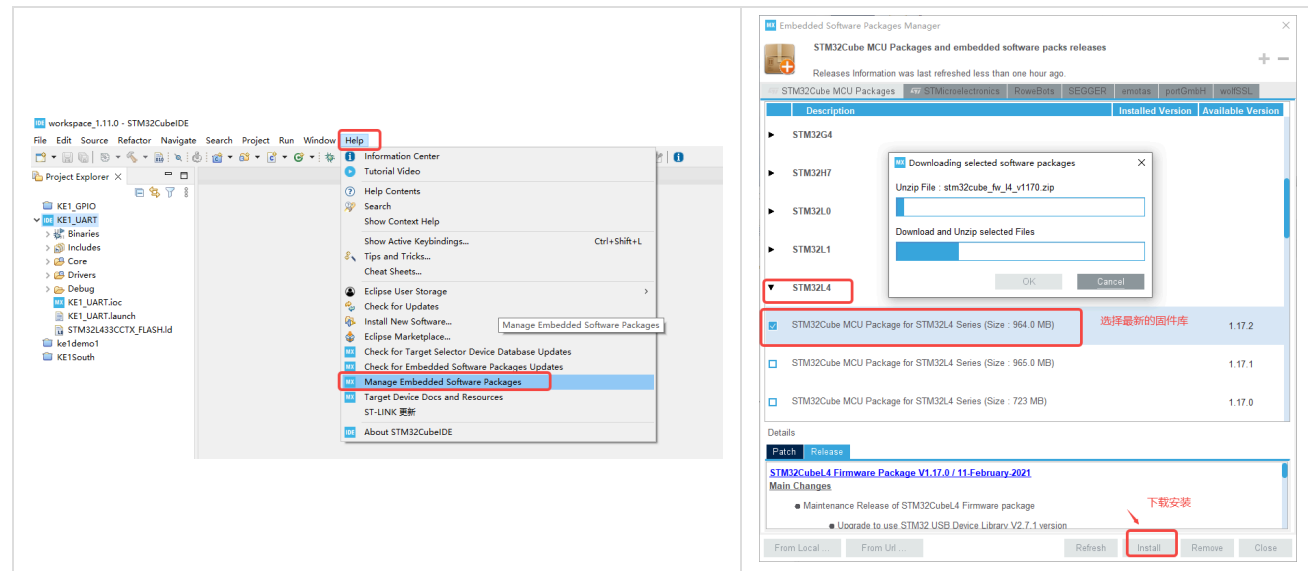
例如 C:\Users\xxx\STM32Cube\Repository\STM32Cube\_FW\_L4\_V1.17.2\Drivers\STM32L4xx\_HAL\_Driver





## 固件源码更新

选择所用MCU型号固件包进行下载更新。例如 KE1 终端使用的MCU型号为 STM32L4xxx，所以选择下载 STM32L4 最新固件包。

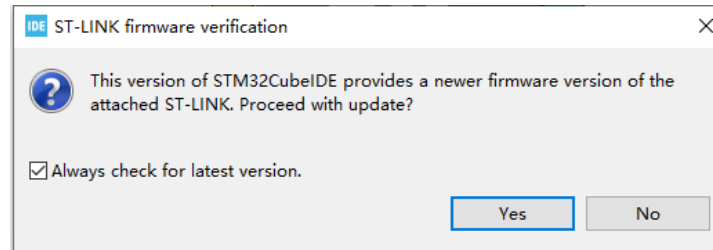


## ST-Link 调试帮助

### ST-Link 调试帮助

本章节记录了在使用STM32CubeIDE进行ST-Link调试时，可能遇到的各种异常错误的解决方法，仅供大家参考。如果我们发现了新的异常也会在本章节进行更新！

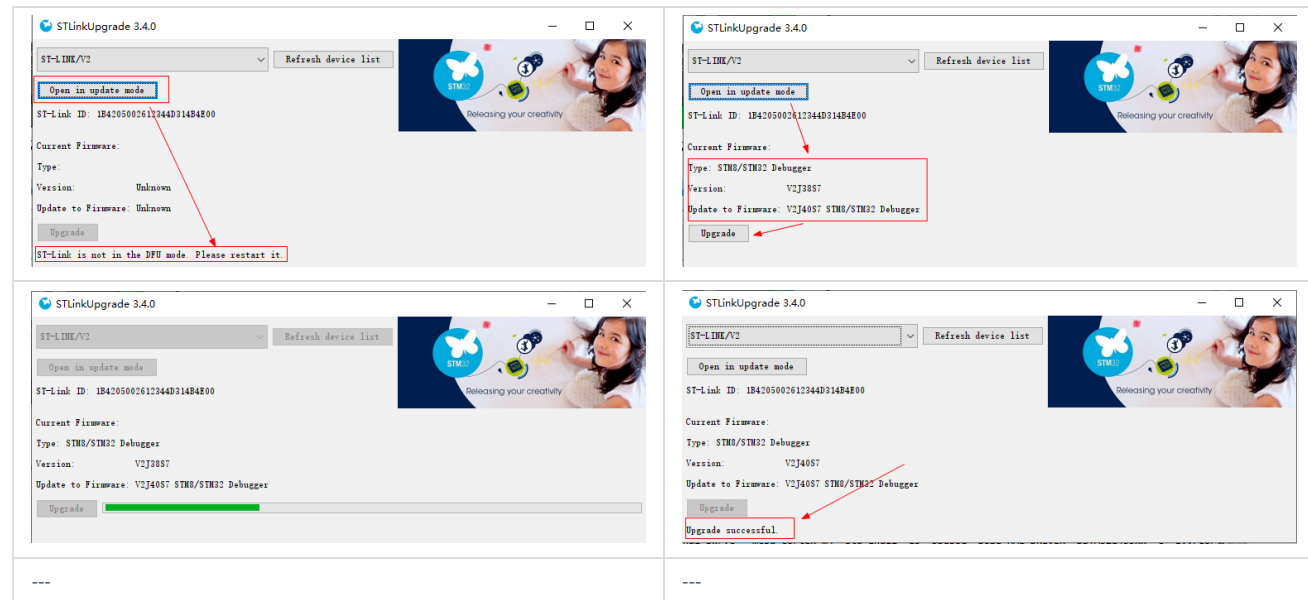
### ST-Link 固件升级



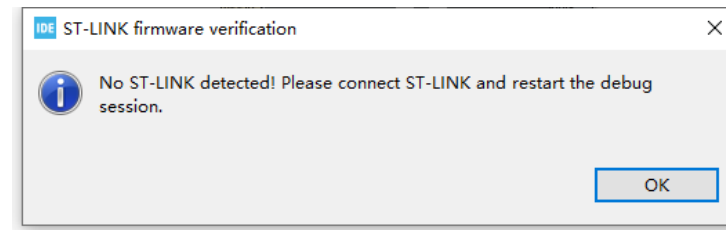
如图提示升级ST-Link固件，为了减少错误，建议大家点击"Yes"进行升级。

步骤如下：

1. 点击“Open in update mode”
2. 重新拔插ST-Link，使其重启
3. 点击“Refresh device list”，可以看到当前的固件信息
4. 点击“Upgrade”开始固件升级，等待成功即可（成功后，可重新拔插一下ST-Link）



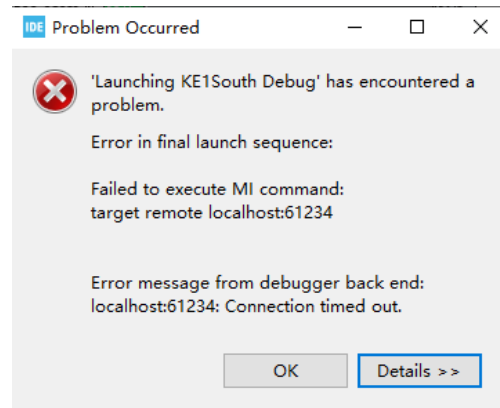
No ST-Link detected!



解决办法:

1. 检查是否开启KE1电源
2. 检查连接线序, 可参考【KE1 快速体验】章节
3. ST-Link连接不稳定, 重新连接一下

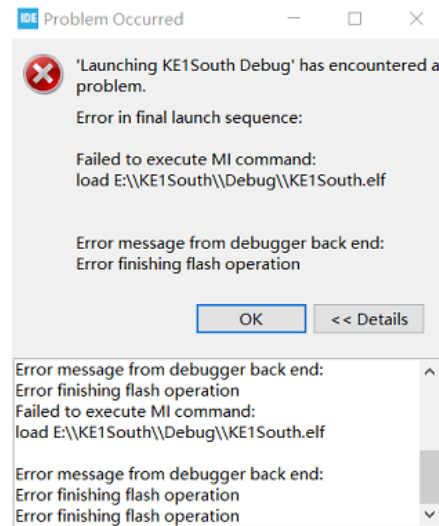
localhost:61234: Connection timed out



原因是：MCU没能正常自动复位

解决办法：一个手指一直按着KE1正下居中的MCU-BOOT键。另外一个手指按一下MCU-RST键，进行手动复位MCU(不要一直按着哦)，然后再次尝试调试。直到正常进入调试界面即可松开所有按键。

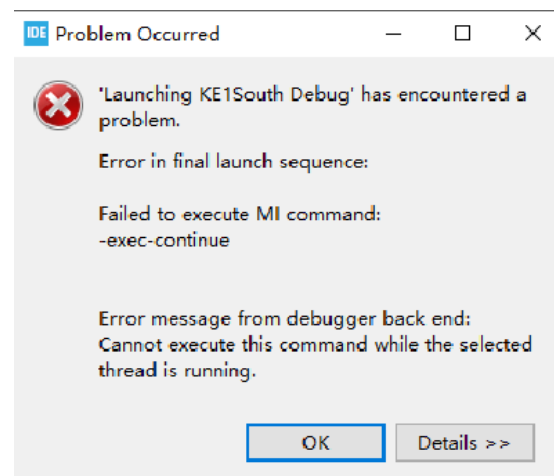
## Error finishing flash operation



原因是：擦除MCU Flash失败，无法复位MCU

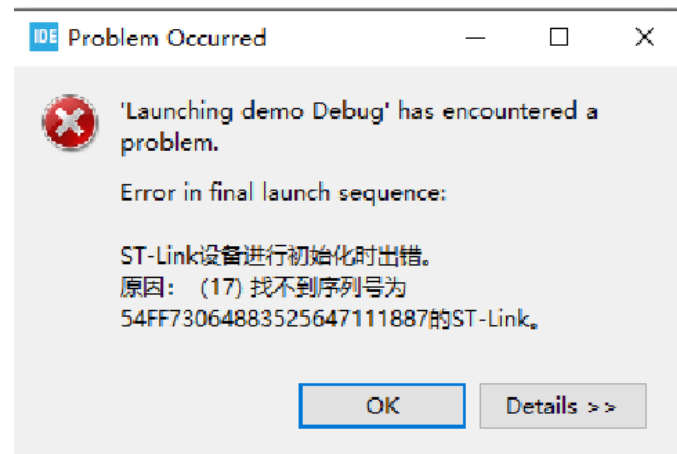
解决办法：一个手指一直按着KE1正下居中的MCU-BOOT键。另外一个手指按一下MCU-RST键，进行手动复位MCU(不要一直按着哦)，然后再次尝试调试。直到正常进入调试界面即可松开所有按键。

## Failed to execute MI command

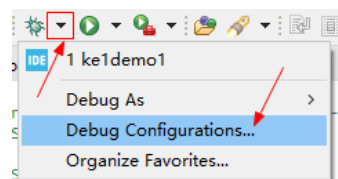


解决办法：重新拔插ST-Link，使其重启

## 找不到序列号为 xxx 的ST-Link



解决办法：在调试器配置界面中，去掉勾选使用特定ST-Link，方法如下图



## IDE Debug Configurations

Create, manage, and run configurations

